TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



CÔNG NGHỆ PHẦN MỀM

Báo cáo bàn giao sản phẩm

BrewLite

Ứng dụng đặt cà phê không dùng tiền mặt

GVHD: ThS. Từ Lãng Phiêu
SV: Cao Cát Lượng - 3123410209
Dương Văn Khánh - 3123410161
Dương Trung Lâm - 3123410190
Đinh Văn Thanh Sơn - 3123410307
Nguyễn Hồng An - 3123410003
Phạm Nguyễn Minh Nhật - 3123410247

TP. HỒ CHÍ MINH, THÁNG 11/2026

Mục lục

1 Phần giới thiệu **4**
- 1.1 Bối cảnh và mục tiêu . . . 4
- 1.2 Phạm vi của sản phẩm . . . 4
- 1.3 Các bên liên quan . . . 4
- 1.4 Công nghệ sử dụng . . . 5
- 1.5 Cấu trúc báo cáo . . . 5

2 Quy trình Agile Scrum **6**
- 2.1 Vai trò, sự kiện và tạo phẩm . . . 6
- 2.2 Product Backlog . . . 6
- 2.3 Kế hoạch Sprint . . . 6
- 2.4 Phân công thành viên . . . 7
- 2.5 Biểu đồ Burndown . . . 7
- 2.6 Definition of Done . . . 7
- 2.7 Sprint Review và Sprint Retrospective . . . 8

3 Phân tích yêu cầu **10**
- 3.1 Yêu cầu chức năng và tiêu chí chấp nhận . . . 10
- 3.2 Yêu cầu phi chức năng và phép kiểm chứng . . . 11
- 3.3 Use-case đặt và thanh toán đơn . . . 12
- 3.4 Giả định, ràng buộc và rủi ro . . . 14
- 3.5 Sơ đồ use-case . . . 15
- 3.6 Yêu cầu nghiệp vụ đặt hàng, giữ kho và hoàn kho . . . 15

4 Thiết kế hệ thống **17**
- 4.1 Kiến trúc tổng thể . . . 17
- 4.2 Mô hình dữ liệu . . . 17
  - 4.2.1 Các thực thể phục vụ quản lý nguyên liệu . . . 18
- 4.3 Thiết kế API . . . 19
  - 4.3.1 Thiết kế xác thực và phân quyền . . . 19
- 4.4 Máy trạng thái đơn hàng (Order State Machine) . . . 22
  - 4.4.1 Vận hành quầy và đồng bộ trạng thái . . . 22
- 4.5 Quy tắc tính giá, giảm giá và điểm thưởng . . . 23
  - 4.5.1 Mã giảm giá và giá trị đơn đã chốt . . . 23
  - 4.5.2 Số dư điểm và sổ giao dịch điểm . . . 24
- 4.6 VNPay Sandbox và xác minh kết quả . . . 24
  - 4.6.1 Phân biệt đơn hàng, lần thanh toán và khoản áp dụng . . . 24
  - 4.6.2 Luồng và các nhánh kết quả . . . 25
- 4.7 Giữ kho và hoàn theo snapshot . . . 25
  - 4.7.1 Định lượng theo size và topping . . . 25
  - 4.7.2 Snapshot: vì sao không hoàn bằng công thức hiện tại? . . . 27
  - 4.7.3 Hủy chủ động và hết thời gian giữ món . . . 27
- 4.8 Đổi món, chuyển tiền và thanh toán chênh lệch . . . 27
  - 4.8.1 Dữ liệu yêu cầu và báo giá . . . 29
  - 4.8.2 Chuyển giá trị đã trả và thu chênh lệch . . . 29
- 4.9 Version trong hồ sơ xử lý và khoản thu dư . . . 29
- 4.10 Mô hình hoạt động, tương tác và miền dữ liệu . . . 31
- 4.11 Thiết kế giao diện . . . 35

5 Hiện thực hệ thống **36**
5.1 Cấu trúc thư mục . . . 36
5.2 Menu, chi tiết sản phẩm và giỏ hàng . . . 36
5.2.1 Tải menu và trạng thái giao diện . . . 36
5.2.2 Tùy chọn và giá hiển thị . . . 36
5.2.3 State giỏ hàng và lưu qua lần tải lại . . . 36
5.2.4 Gửi đặt hàng và khôi phục khi mất phản hồi . . . 37
5.3 Xác thực . . . 38
5.3.1 Đăng ký, đăng nhập và dữ liệu công khai . . . 38
5.3.2 Chuỗi kiểm tra JWT, vai trò và chủ đơn . . . 38
5.3.3 Trạng thái đăng nhập ở giao diện . . . 38
5.4 Tạo đơn và tích hợp thanh toán . . . 38
5.4.1 Hiện thực VNPay và bảo đảm ghi nhận một lần . . . 38
5.5 Toàn vẹn đơn hàng, thanh toán và tồn kho . . . 40
5.5.1 Order State Machine . . . 40
5.5.2 Thanh toán idempotent . . . 40
5.5.3 Kiểm soát tồn kho khi đặt đồng thời . . . 42
5.5.4 Khuyến mãi và điểm thưởng . . . 49
5.6 Hiện thực đổi món sau thanh toán . . . 49
5.6.1 Một quyết định, một transaction . . . 49
5.6.2 Tranh chấp với pha chế và thao tác quầy . . . 49
5.6.3 Bảo toàn tiền đã chuyển . . . 49
5.7 Hiện thực quầy và cập nhật theo thời gian thực . . . 52
5.7.1 Kiểm soát chuyển trạng thái tại backend . . . 52
5.7.2 Thông báo sau commit và tải lại dữ liệu . . . 52
5.8 Đóng gói và triển khai . . . 52

6 Kiểm thử và kết quả **54**
6.1 Chiến lược kiểm thử . . . 54
6.2 Kiểm thử các quy tắc toàn vẹn nghiệp vụ . . . 54
6.3 Danh sách test case tự động . . . 54
6.4 Kiểm chứng luồng đặt hàng, giữ kho và hoàn kho . . . 55
6.4.1 Kết quả kiểm chứng theo phạm vi . . . 56
6.5 Kiểm chứng thanh toán VNPay . . . 56
6.6 Kiểm chứng đổi món và thu chênh lệch . . . 58
6.7 Kiểm chứng xác thực và phân quyền . . . 59
6.8 Kiểm chứng mã giảm giá và điểm thưởng . . . 60
6.9 Kiểm chứng thao tác quầy và realtime . . . 60
6.10 Tiêu chí kiểm chứng menu, tùy chọn và giỏ hàng . . . 62
6.11 Kiểm thử chức năng thủ công . . . 62

7 Hướng dẫn triển khai **63**
7.1 Chạy bằng Docker . . . 63
7.2 Chạy chế độ phát triển . . . 63
7.3 Biến môi trường . . . 63
7.4 Dữ liệu mẫu . . . 64

8 Đánh giá và hướng phát triển **64**
8.1 Giá trị của thiết kế 64
8.2 Định hướng mở rộng 64

9 Kết luận **64**

1 Phần giới thiệu

1.1 Bối cảnh và mục tiêu

BrewLite là ứng dụng web cho phép khách hàng đặt đồ uống và thanh toán không dùng tiền mặt chỉ trong vài thao tác, nhằm giảm thời gian xếp hàng và nhận đơn nhanh tại quầy. Tầm nhìn sản phẩm (Product Vision) là: "BrewLite giúp khách hàng đặt và thanh toán đồ uống không dùng tiền mặt chỉ trong vài chạm, giảm thời gian xếp hàng và nhận đơn nhanh tại quầy."

Nhóm phát triển BrewLite theo quy trình Agile Scrum, từ thu thập yêu cầu, xây dựng kịch bản đến phát triển, kiểm thử và bàn giao sản phẩm. Công nghệ được lựa chọn là **NestJS** cho phần Backend và **Next.js** cho phần Frontend.

1.2 Phạm vi của sản phẩm

Sản phẩm tối thiểu (MVP) trải dài từ lúc khách chọn sản phẩm đến lúc thanh toán và xác nhận đơn hàng.

- **Trong phạm vi:** xem menu, chọn size và topping, giỏ hàng, đăng ký và đăng nhập, đặt hàng, thanh toán giả lập và VNPay Sandbox, xác nhận và lịch sử đơn hàng, hủy đơn, mã giảm giá, điểm thưởng, đóng gói bằng Docker.
- **Ngoài phạm vi:** vận hành giao nhận bằng giao diện riêng và chuyển tiền ngân hàng thật tự động. Barista tiếp nhận đơn và xử lý nghiệp vụ thanh toán trong phạm vi quán nhỏ; giao hàng được đề cập ở mức nghiệp vụ.

1.3 Các bên liên quan

Bên liên quan	**Vai trò**
Khách hàng	Đặt đồ uống, chọn size và topping, thanh toán không tiền mặt, theo dõi trạng thái đơn.
Nhân viên quầy (Barista)	Tiếp nhận đơn và cập nhật trạng thái pha chế. Có giao diện nhận đơn, pha chế, duyệt đổi món và theo dõi khoản thu dư.
Quản lý cửa hàng / Product Owner	Xác định yêu cầu, ưu tiên Product Backlog và nghiệm thu sản phẩm sau mỗi Sprint.
Giảng viên nghiệm thu	Đánh giá quy trình, sản phẩm và mức độ đáp ứng tiêu chí chấp nhận.
Nhóm phát triển	Phân tích, thiết kế, lập trình, kiểm thử và bàn giao hệ thống.
Đơn vị thanh toán	Cổng thanh toán Ví hoặc Thẻ, được mô phỏng hoặc tích hợp VNPay Sandbox trong phạm vi kiểm thử.

Bảng 1: Các bên liên quan

Thành phần	Công nghệ
Frontend	Next.js (React, TypeScript), TailwindCSS, Zustand (quản lý giỏ hàng và đăng nhập)
Backend	NestJS (TypeScript), REST, class-validator, JWT (Passport), bcrypt
Cơ sở dữ liệu	PostgreSQL 16, Prisma ORM phiên bản 7
Thanh toán	Mock payment và tích hợp VNPay Sandbox; không chứng minh giao dịch tiền thật
Kiểm thử	Vitest, Supertest (test chạy trên cơ sở dữ liệu thật)
Đóng gói và quản lý mã nguồn	Docker, Docker Compose, Git và GitHub

Bảng 2: Công nghệ sử dụng

1.4 Công nghệ sử dụng

1.5 Cấu trúc báo cáo

Phần 2 trình bày quy trình Scrum mà nhóm áp dụng. Phần 3 phân tích yêu cầu. Phần 4 mô tả thiết kế hệ thống. Phần 5 mô tả quá trình hiện thực. Phần 6 trình bày kiểm thử và kết quả. Phần 7 hướng dẫn triển khai. Phần 8 đánh giá, hạn chế và hướng phát triển. Phần 9 là kết luận.

2 Quy trình Agile Scrum

2.1 Vai trò, sự kiện và tạo phẩm

Nhóm yếu tố	**Thành phần**
Vai trò (Roles)	Product Owner (PO), Scrum Master (SM), Development Team
Sự kiện (Events)	Sprint Planning, Daily Scrum (15 phút mỗi ngày), Sprint Review, Sprint Retrospective
Tạo phẩm (Artifacts)	Product Backlog, Sprint Backlog, Increment (sản phẩm tăng dần)

Bảng 3: Các thành phần của Scrum

2.2 Product Backlog

Product Backlog gồm 10 hạng mục phát triển, mỗi hạng mục có tiêu chí chấp nhận để kiểm tra kết quả bàn giao.

#	**Hạng mục**	**Tiêu chí chấp nhận chính**
1	Khởi tạo dự án và cấu trúc	Hai thư mục frontend và backend chạy được, có README, Git, file `.env.example`
2	API danh sách sản phẩm	`GET /products` trả JSON menu (id, tên, giá, ảnh)
3	Trang Menu (Frontend)	Gọi API, hiển thị lưới sản phẩm, có trạng thái loading và empty
4	Chi tiết và tùy chọn sản phẩm	Chọn size (S, M, L) và topping, tính giá theo tùy chọn
5	Giỏ hàng	Thêm, sửa số lượng, xóa; lưu state; tự tính tổng tiền; badge số lượng
6	API tạo đơn hàng	`POST /orders` nhận giỏ hàng, kiểm tra dữ liệu, lưu đơn trạng thái PENDING, trả mã đơn
7	Đăng ký và đăng nhập (JWT)	Băm mật khẩu bằng bcrypt, phát JWT, guard bảo vệ route đặt đơn
8	Thanh toán không tiền mặt	Mock payment: chọn Ví hoặc Thẻ; đơn chuyển PAID khi thành công, PAYMENT_FAILED khi lỗi
9	Xác nhận, lịch sử đơn và bàn giao	Màn hình xác nhận, `GET /orders/me`, docker-compose chạy cả 3 thành phần, README, demo từ đầu đến cuối
10	Nghiệp vụ backend	State machine đơn hàng, thanh toán idempotent, kiểm soát tồn kho khi đặt đồng thời, khuyến mãi và điểm thưởng

Bảng 4: Product Backlog của BrewLite

2.3 Kế hoạch Sprint

Nhóm chia công việc thành 11 **chặng** nhỏ (mỗi chặng là một bước tăng dần của sản phẩm, được đặt tên để lập kế hoạch, chưa có minh chứng tag Git tương ứng), ước lượng **điểm khối lượng** cho từng chặng theo ba tiêu chí là lượng code, độ khó khái niệm và công sức kiểm thử, rồi chia lại 3 Sprint sao cho khối lượng gần nhau.

Hai ngày cuối (ngày 13 và 14) dùng để kiểm tra toàn bộ hệ thống trên bản sao sạch của kho mã nguồn, hoàn thiện tài liệu, tập demo và làm ngày dự phòng nếu có chặng bị trễ.

Chặng	Nội dung	Sprint	Điểm
chang-0	Schema Prisma, dữ liệu mẫu, Docker cho cơ sở dữ liệu	1	6
chang-1	API GET /products và trang Menu	1	5
chang-2	Chi tiết sản phẩm, chọn size và topping	1	7
chang-3	Giỏ hàng	1	7
chang-4	Đăng ký, đăng nhập JWT	2	14
chang-5	Tạo đơn và thanh toán	2	11
chang-6a	State machine, thanh toán idempotent, bộ test Vitest	2	16
chang-6b	Tồn kho bằng khóa lạc quan	3	10
chang-6c	Tách bước đặt hàng và thanh toán, hủy đơn	3	6
chang-6d	Mã giảm giá và điểm thưởng	3	12
chang-7	Docker Compose và README	3	6
		Tổng	**100**

Bảng 5: Các chặng và điểm khối lượng

Sprint	Ngày	Mục tiêu (Sprint Goal)	Chặng	Điểm
1	1 đến 5	Nền tảng, hiển thị menu và giỏ hàng	0, 1, 2, 3	25
2	6 đến 8	Đăng nhập, đặt đơn và thanh toán an toàn	4, 5, 6a	41
3	9 đến 12	Tồn kho, tách luồng đặt hàng, khuyến mãi và bàn giao	6b, 6c, 6d, 7	34

Bảng 6: Kế hoạch 3 Sprint trong 14 ngày

2.4 Phân công thành viên

Bảng dưới giữ phân công dự kiến của nhóm; điểm ước lượng không phải số đo đóng góp thực tế. Cần bổ sung biên bản và liên kết commit/PR để xác nhận kết quả thực hiện.

Nhóm	Chặng	Điểm	Tỷ lệ	Thành viên
A. Nền tảng Project, tài liệu mẫu, giao diện sản phẩm đầu	0, 1, 7	17	17%	Cao Cát Lượng
B. Chi tiết sản phẩm và tính toán tồn kho	2, 6b	17	17%	Nguyễn Hồng An
C. Đăng nhập, đăng ký, JWT	4	14	14%	Dương Trung Lâm
D. Giỏ hàng và tạo đơn	3, 5	18	18%	Đinh Văn Thanh Sơn
E. Thanh toán và state machine	6a	16	16%	Phạm Nguyễn Minh Nhật
F. Hủy đơn, mã giảm giá, điểm thưởng, giả lập vnpay	6c, 6d	18	18%	Dương Văn Khánh

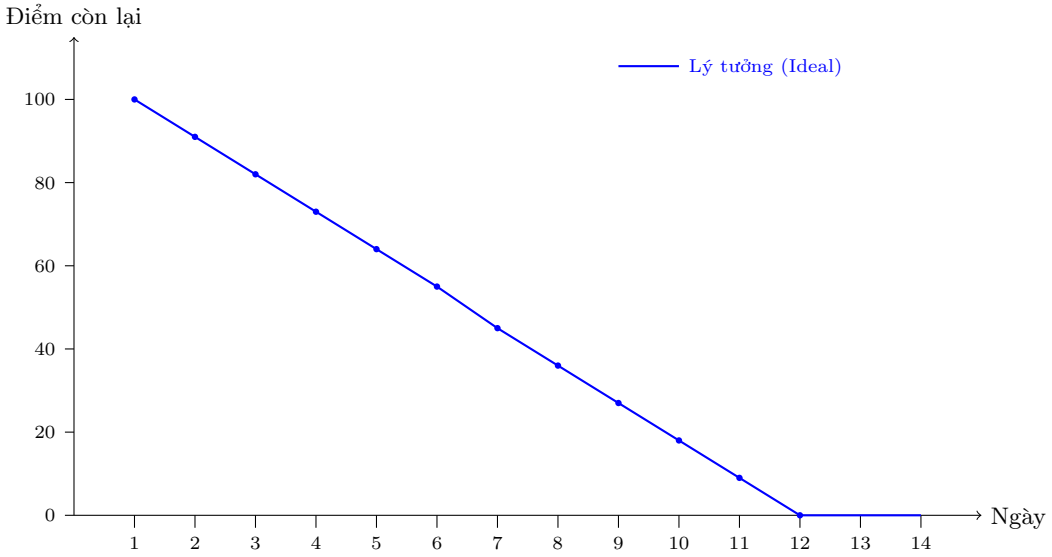
Bảng 7: Phân công thành viên theo nhóm công việc

2.5 Biểu đồ Burndown

Biểu đồ Burndown theo dõi khối lượng công việc còn lại (tính bằng điểm) giảm dần theo từng ngày. Đường lý tưởng (Ideal) được tính từ kế hoạch ở trên; đường thực tế (Actual) phản ánh tiến độ thật của nhóm.

2.6 Definition of Done

Một hạng mục được xem là hoàn thành khi thỏa các điều kiện sau:



Hình 1: Đường Burndown lý tưởng cho toàn dự án (100 điểm ước lượng, 12 ngày làm việc)

Ngày	1	2	3	4	5	6	7	8	9	10	11	12
Lý tưởng	100	91	82	73	64	55	45	36	27	18	9	0
Thực tế	Điền số điểm còn lại cuối mỗi ngày											

Bảng 8: Số điểm còn lại cuối mỗi ngày

- Code chạy được, không lỗi build, đúng tiêu chí chấp nhận của task.
- Có commit trên Git với mô tả rõ ràng, được xem lại (review) trong nhóm.
- API có kiểm tra dữ liệu đầu vào; giao diện xử lý trạng thái loading và lỗi cơ bản.
- Có hướng dẫn chạy trong README.

2.7 Sprint Review và Sprint Retrospective

Các bài học kỹ thuật đã gặp trong quá trình thực hiện, có thể dùng làm gợi ý cho phần Retrospective:

- Phiên bản mới của NestJS phát hành các gói theo chuẩn ES Module nên bộ test Jest không nạp được; nhóm chuyển sang Vitest (kèm `unplugin-swc` để giữ metadata của decorator).
- Lệnh cài đặt mặc định kéo về Prisma phiên bản 8 (bản thử nghiệm) có cách cấu hình khác hẳn; nhóm cố định phiên bản 7.
- Quy tắc giá size, topping và quy tắc hủy đơn đã thanh toán cần được thống nhất sớm, nhóm phải tự thống nhất, đáng lẽ nên chốt với Product Owner từ Sprint 1.
- Ban đầu nút "thanh toán" tạo đơn và trả tiền trong một thao tác nên trạng thái PENDING gần như không tồn tại; nhóm tách thành hai bước để tách tạo đơn khỏi xác nhận thanh toán và hỗ trợ hủy đơn chưa trả.

- Seed dữ liệu chèn id gõ tay làm lệch bộ đếm id của PostgreSQL, gây lỗi khi test tạo sản phẩm mới; khắc phục bằng cách đặt lại bộ đếm sau khi seed.

3 Phân tích yêu cầu

3.1 Yêu cầu chức năng và tiêu chí chấp nhận

Các yêu cầu được ưu tiên theo MoSCoW: Must là bắt buộc, Should là ưu tiên cao, Could là tùy chọn; Won't nằm ngoài phạm vi bàn giao. Tiêu chí dưới đây là căn cứ nghiệm thu, kết quả kiểm chứng được trình bày riêng ở phần kiểm thử.

Bảng 9: Yêu cầu chức năng và tiêu chí chấp nhận

Mã	**Yêu cầu**	**Ưu tiên**	**Tiêu chí chấp nhận có thể kiểm chứng**
FR-01	Xem menu	Must	AC-01: GET /products trả danh sách có id, tên, giá, ảnh; UI có loading, empty và error; lỗi tải không làm mất điều hướng
FR-02	Xem chi tiết và tùy chọn	Must	AC-02: GET /products/:id trả chi tiết/options; chọn S/M/L, topping cập nhật giá; id không tồn tại có trạng thái không tìm thấy
FR-03	Giỏ hàng	Must	AC-03: thêm/gộp đúng tùy chọn, đổi số lượng, xóa; tổng bằng tổng dòng; reload còn giỏ; giỏ rỗng không gửi đơn
FR-04	Đăng ký, đăng nhập và JWT	Must	AC-04: đăng ký email duy nhất; sai mật khẩu không cấp token; API bảo vệ thiếu/sai JWT trả 401; response không lộ passwordHash
FR-05	Tạo đơn và tính giá ở backend	Must	AC-05: input hợp lệ trả Order PENDING; tiền từ quy tắc backend; sai size/topping/qty bị từ chối không đổi database; cùng key/nội dung không tạo thêm đơn
FR-06	Thanh toán Ví/Thẻ mô phỏng	Must	AC-06: khi chỉ bật mock, thành công ghi Payment/PAID; lỗi có kết quả PAYMENT_FAILED; mock bị chặn khi VNPay bật để không bỏ qua cổng
FR-07	Xác nhận, theo dõi và lịch sử	Must	AC-07: chủ đơn xem đúng mã/trạng thái/món/tiền; GET /orders/me không trả đơn người khác; refresh/realtime không nhân đôi dữ liệu
FR-08	Barista quản lý trạng thái	Must	AC-08: chỉ BARISTA gọi staff API; chuyển trạng thái đúng máy trạng thái đơn hàng; không pha đơn thiếu tiền hoặc có REQUESTED; trạng thái cũ bị từ chối khi thao tác đồng thời
FR-09	Giữ kho đồng thời và hủy an toàn độ đúng nghiệp vụ	Should	AC-09: hai khách tranh phần cuối chỉ một đơn giữ thành công; kho không âm; định lượng đúng size/topping; hủy hợp lệ hoàn snapshot một lần dù Recipe đã đổi; đơn hết hạn giữ món được đóng và thông báo đặt lại; đơn đã pha không được tự hủy
FR-10	Voucher và điểm thưởng	Should	AC-10: mã hết hạn/inactive bị từ chối; giảm và điểm theo chính sách giảm giá và tích điểm; callback lặp không cộng lại điểm
FR-11	VNPay URL và QR	Should	AC-11: backend tạo URL ký đúng số tiền còn phải trả; QR mở cùng URL và có nút mở trang; URL hết hạn/đơn không payable không hiển thị QR khả dụng
FR-12	Kết quả cổng an toàn	Should	AC-12: sai chữ ký/merchant/amount không đổi PAID; Return chỉ chuyển hướng; IPN/QueryDr hợp lệ cập nhật đúng một lần trong transaction
FR-13	Thanh toán lại, kết quả muộn và thu trùng	Should	AC-13: cùng key trả cùng lần; mã còn hạn tái dùng; sau hết hạn QR và đơn còn hạn giữ món có thể cấp lần mới, không xóa lần cũ; hai lần thành công chỉ một lần áp dụng cho đơn và khoản dư có hồ sơ xử lý
FR-14	Khách yêu cầu đổi món	Should	AC-14: liên kết hỗ trợ nhỏ trên đơn PAID mở menu/giỏ riêng; chọn size/topping bình thường; báo giá từ backend; cùng key không tạo trùng; REQUESTED có thể rút bằng version

Mã	Yêu cầu	Ưu tiên	Tiêu chí chấp nhận có thể kiểm chứng
FR-15	Quầy duyệt đổi và thu chênh lệch	Should	AC-15: giá đã xác nhận giữ theo snapshot; duyệt chuyển credit/đổi giữ kho nguyên tử; tiền thu mới = total - credit; thấp hơn giá trị cũ bị từ chối; hai người cùng duyệt chỉ một thành công
FR-16	Theo dõi hoàn khoản thu không áp dụng	Should	AC-16: thu dư mở một hồ sơ theo attempt; BARISTA nhận xử lý, nhập chứng từ rồi hoàn tất theo version; chứng từ trùng trả lỗi; khách không tự xác nhận hoàn
FR-17	Hàng đợi hỗ trợ tại quầy	Should	AC-17: queue lọc trạng thái, tìm mã/email, phân trang 20 dòng; xem món cũ/mới, tiền và ghi chú; thiếu quyền 403; page/status sai 400
FR-18	Đóng gói và chạy local	Must	AC-18: clone sạch, điền biến mẫu, một lệnh Compose chạy web/API/DB; migration+seed trên DB sạch tạo được dữ liệu; health endpoint phản ánh tình trạng dịch vụ
FR-19	CI/CD và bàn giao	Must	AC-19: PR tự install/lint/test/build; failed check không merge; release tạo image có version/SHA, deploy, migrate an toàn, smoke test; có cách rollback và URL HTTPS frontend/API/health
FR-20	Giao hàng	Won't	AC-20: phạm vi bàn giao ghi rõ chưa có UI tài xế/điều phối/tracking; không quảng bá trạng thái hiện tại là vòng đời giao hàng đầy đủ

3.2 Yêu cầu phi chức năng và phép kiểm chứng

Các ngưỡng là mục tiêu nghiệm thu, chưa phải số liệu đo đã đạt.

Bảng 10: NFR và phép kiểm chứng

Mã	Yêu cầu	Ưu tiên	Tiêu chí chấp nhận / phép đo
NFR-01	Bảo mật xác thực/phân quyền	Must	Mật khẩu băm; khách không đọc đơn người khác/gọi staff; 401/403/404 đúng hợp đồng; không có secret trong Git/log/ảnh
NFR-02	Toàn vẹn tiền và kho	Must	Chạy test cạnh tranh/idempotency/rollback: không kho âm, không nhân đôi settlement/điểm, không chuyển credit vượt tổng; mọi lỗi giữa transaction không để dữ liệu dở dang
NFR-03	Hiệu năng API nội bộ	Should	Với 50 sản phẩm, 20 người dùng đồng thời, sau warm-up: p95 GET sản phẩm và GET đơn $\leq$500ms trên môi trường ghi rõ CPU/RAM/DB; tách độ trễ gateway khỏi số liệu; ghi tỷ lệ lỗi và lệnh đo
NFR-04	Mobile-first và khả năng truy cập	Must	Ở 390px và 1440px không tràn ngang; nhãn form, focus bàn phím, thông báo lỗi và trạng thái chờ rõ; nút chính tối thiểu 44px; xác nhận tiền không chỉ dựa màu sắc
NFR-05	Tính khả dụng và phục hồi	Must	Có loading/empty/error, retry có kiểm soát; mất Return không mất kết quả đã ghi; không treo UI vô hạn; health check production dùng cho smoke
NFR-06	Tái tạo và triển khai	Must	Install từ lockfile; build frontend/backend; migration+seed trên database mới; Compose đủ 3 dịch vụ; HTTPS và DB production không public trực tiếp
NFR-07	Quan sát và hỗ trợ	Must	Log có thời gian và tham chiếu lỗi, không lộ token/secret; phân biệt Order/attempt/refund; có hướng dẫn xử lý lỗi thanh toán; health không trả cấu hình nhạy cảm
NFR-08	Bảo trì và truy vết	Must	Mỗi FR/NFR liên kết use-case, thiết kế, code/test; sơ đồ có nguồn+render; quyết định có ADR; không có đường dẫn/file minh chứng giả

Mã	Yêu cầu	Ưu tiên	Tiêu chí chấp nhận / phép đo
NFR-09	Quality gate	Must	CI tự chạy trên PR, branch protection yêu cầu checks; lưu link/log một lần thành công và một lần phát hiện lỗi; test không dùng database production
NFR-10	Sao lưu, migration và khôi phục	Must	Backup trước migration; thử khôi phục DB riêng; script deploy không reset hoặc seed đè DB đang dùng; runbook rollback nêu phần app và phần dữ liệu
NFR-11	Phiên đăng nhập nhất quán	Should	Chính sách một tài khoản/origin hoặc phiên riêng/tab được chốt; đổi tài khoản/logout không để response cũ ghi đè user/token mới; có test hai tab và reload
NFR-12	Cài đặt PWA	Could	Có manifest, icon và kiểm tra cài đặt nếu nhóm chọn triển khai PWA; không coi hỗ trợ mobile là bằng chứng đã có offline/PWA

3.3 Use-case đặt và thanh toán đơn

Bảng 11: Thông tin UC-01

Thuộc tính	Nội dung
Actor chính	Khách hàng
Actor hỗ trợ	Dịch vụ thanh toán; barista nhận đơn đủ tiền và hỗ trợ khi có bất thường
Trigger	Khách bấm Đặt hàng từ giỏ có món, sau đó chủ động bắt đầu thanh toán
Mục tiêu	Có một đơn đúng giá/tùy chọn, giữ nguyên liệu hợp lệ và xác định kết quả thu tiền; khách nhận được mã/trạng thái đơn
Tiền điều kiện	Menu và cấu hình giá tồn tại; khách đăng nhập trước khi gửi tạo đơn; PostgreSQL sẵn sàng; gateway mock hoặc VNPay được cấu hình theo chế độ
Hậu điều kiện thành công	Order PAID, reservation còn giữ; đúng một khoản thanh toán áp dụng cho đơn; điểm trên tiền mới được ghi cùng transaction; đơn có thể xuất hiện tại quầy
Hậu điều kiện tối thiểu	Không kho âm/đơn nửa chừng/thu lặp do callback; giao dịch chưa rõ giữ lịch sử để đối soát; frontend không tự coi timeout là thất bại chắc chắn
Liên kết yêu cầu	FR-01–08, FR-09–13; NFR-01/02/04/05/07/11
Quy tắc	Giá do backend chốt; giữ/hoàn theo snapshot; thanh toán xác nhận một lần; chỉ chuyển trạng thái hợp lệ

Bảng 12: Luồng chính UC-01

Bước	Khách / Next.js UI	NestJS / database / dịch vụ thanh toán
M1	Mở menu, xem chi tiết	GET /products và /products/:id trả sản phẩm và tùy chọn
M2	Chọn size/topping/số lượng, thêm giỏ	Chưa có Order, chưa khóa/giữ kho
M3	Mở giỏ, xác nhận món, voucher nếu có	UI tính tiền tham khảo; backend mới là nguồn giá chính thức
M4	Đăng nhập nếu cần, gửi tạo đơn với key cho lần đặt	JWT và DTO được kiểm tra; key được nhận trước khi giữ nguyên liệu
M5	Chờ kết quả tạo đơn	Transaction tính lại tiền/voucher, khóa nguyên liệu có thứ tự, kiểm tra hàng, trừ lượng giữ, tạo Order PENDING và snapshot; lỗi thì rollback
M6	Nhận mã đơn và bấm Thanh toán	Backend xác thực chủ đơn, trạng thái và snapshot; khóa Order; tạo/tái dùng attempt theo key; số tiền từ total - creditApplied

Bước	Khách / Next.js UI	NestJS / database / dịch vụ thanh toán
M7	Quét QR mở URL hoặc bấm mở trang VNPay	VNPay Sandbox nhận URL ký; mock thực hiện theo chế độ kiểm thử riêng khi VNPay tắt
M8	Thực hiện xác nhận trên cổng	IPN đến backend; backend xác minh chữ ký, merchant/ref/amount/status trước khi nhận kết quả
M9	Có thể quay về trang đơn qua Return	Return chuyển về giao diện; trình duyệt đọc trạng thái backend. IPN/QueryDr mới xác nhận Payment, PAID và điểm trong transaction
M10	Xem xác nhận, lịch sử và tiến độ	UI đọc trạng thái từ backend, cập nhật qua HTTP/realtime; barista bắt đầu pha nếu đơn đủ điều kiện

Bảng 13: Luồng thay thế UC-01

Mã / điểm rẽ	Luồng	Kết quả
A1 tại M4	Chưa đăng nhập → mở đăng nhập với next phù hợp → quay về giỏ	Không tạo đơn trước xác thực
A2 tại M4–M5	Mất response → gửi lại cùng key/nội dung	Trả lại cùng Order; không trừ kho thêm
A3 tại M6	Cùng key tạo thanh toán hoặc key khác khi mã còn hiệu lực	Trả/tái dùng attempt còn hiệu lực, không tạo nhiều mã hoạt động do bấm lặp
A4 tại M7–M9	Xác nhận giao dịch thất bại	Lưu kết quả; đơn có thể PAYMENT_FAILED, nguyên liệu chưa tự hoàn chỉ vì thanh toán lỗi; khách retry hoặc hủy nếu đủ điều kiện
A5 tại M7	Mã hết hạn, chưa có khoản thu đã xác minh	Backend thử QueryDr, có thể cấp attempt mới sau hết hạn; giữ lần cũ để nhận kết quả muộn, cảnh báo khách nguy cơ thu trùng
A6 tại M9	Return đến trước IPN hoặc khách đóng trình duyệt	Đơn chưa tự PAID từ Return; mở lịch sử/kiểm tra kết quả, IPN/QueryDr có thể cập nhật sau
A7 sau M5	Khách muốn hủy đơn chưa trả	Kiểm tra đơn chưa thanh toán và còn được hủy; nếu hợp lệ hủy và hoàn snapshot cùng transaction
A8 sau M10	Khách cần đổi món chưa pha	Chuyển UC-09; không hủy đơn cũ khi khách mới mở menu đổi

Bảng 14: Ngoại lệ UC-01

Mã	Điều kiện	Xử lý và dữ liệu cần giữ
E1	Size/topping/qty/product/voucher không hợp lệ	Trả lỗi có thể hiểu; không tạo đơn/giữ kho dở dang
E2	Hai khách tranh lượng cuối, người sau không đủ nguyên liệu	Một transaction thành công, transaction thiếu hàng rollback; thông báo thiếu món; không stock âm
E3	Cùng key tạo đơn nhưng khác nội dung	409, yêu cầu kiểm tra đơn trước; không tự dùng key đó cho đơn khác
E4	Sai JWT hoặc không sở hữu đơn	Từ chối theo guard/quyền; không trả dữ liệu người khác

Mã	Điều kiện	Xử lý và dữ liệu cần giữ
E5	Callback sai chữ ký/ref/merchant/amount	Không cập nhật PAID/Payment/điểm; phản hồi theo giao thức cổng
E6	QueryDr timeout/không có kết quả chắc chắn	Giữ trạng thái chưa rõ; không suy ra ngân hàng đã trả tiền; UI có đường kiểm tra/lịch sử/hỗ trợ
E7	Callback lặp, đồng thời hoặc đến sai thứ tự	Unique/khóa Order/idempotency; không hạ thanh toán đã xác nhận thành thất bại do callback cũ
E8	Lỗi ghi Payment hoặc điểm	Rollback transaction; callback có thể được gửi lại và xử lý an toàn
E9	Hai attempt đều thu thành công	Một khoản áp dụng cho đơn, khoản còn lại có REVIEW/paidAt để quầy xử lý hoàn khoản thu dư
E10	Tiền về sau khi đơn đã đóng/không còn nguyên liệu giữ hợp lệ	Không tự pha/trừ kho lại; giữ khoản thu và hồ sơ hỗ trợ
E11	Ngrok/Return cấu hình sai hoặc cảnh báo trình duyệt	Return có thể chưa về UI; kết quả phải kiểm tra từ backend; không thanh toán lại chỉ vì chưa chuyển trang

Bảng 15: Kịch bản nghiệm thu UC-01

ID	Tiền đề / bước kiểm tra	Mong đợi	Yêu cầu
UC01-T01	Menu → tùy chọn → giỏ → đăng nhập → tạo → mock thành công → xác nhận	Một đơn, giá đúng, PAID, có lịch sử	FR-01–07
UC01-T02	Hai yêu cầu đồng thời mua phần cuối	Một thành công, một thiếu hàng; kho không âm	FR-09, NFR-02
UC01-T03	Gửi lại tạo đơn và IPN nhiều lần	Một Order cho key, một Payment áp dụng, điểm một lần	FR-05/12
UC01-T04	Mock thất bại rồi retry; hủy đơn chưa trả nếu đủ điều kiện	Trạng thái/giữ kho/hoàn snapshot theo điều kiện hủy đơn chưa thanh toán	FR-06/09
UC01-T05	VNPay Return hợp lệ nhưng chưa IPN	Không tự PAID; giao diện chờ/kiểm tra	FR-12
UC01-T06	VNPay IPN sai chữ ký hoặc sai tiền	Không ghi nhận thanh toán	FR-12, NFR-01
UC01-T07	Mã cũ hết hạn, query lỗi, mã mới được cấp, cả hai IPN thành công	Một lần phục vụ, khoản dư có hồ sơ	FR-13/16
UC01-T08	Đổi 45.000đ sang 75.000đ, duyệt rồi thu thêm	UI nổi bật 30.000đ; backend chỉ thu 30.000đ; không trừ kho lần hai khi trả thêm	FR-14/15

3.4 Giả định, ràng buộc và rủi ro

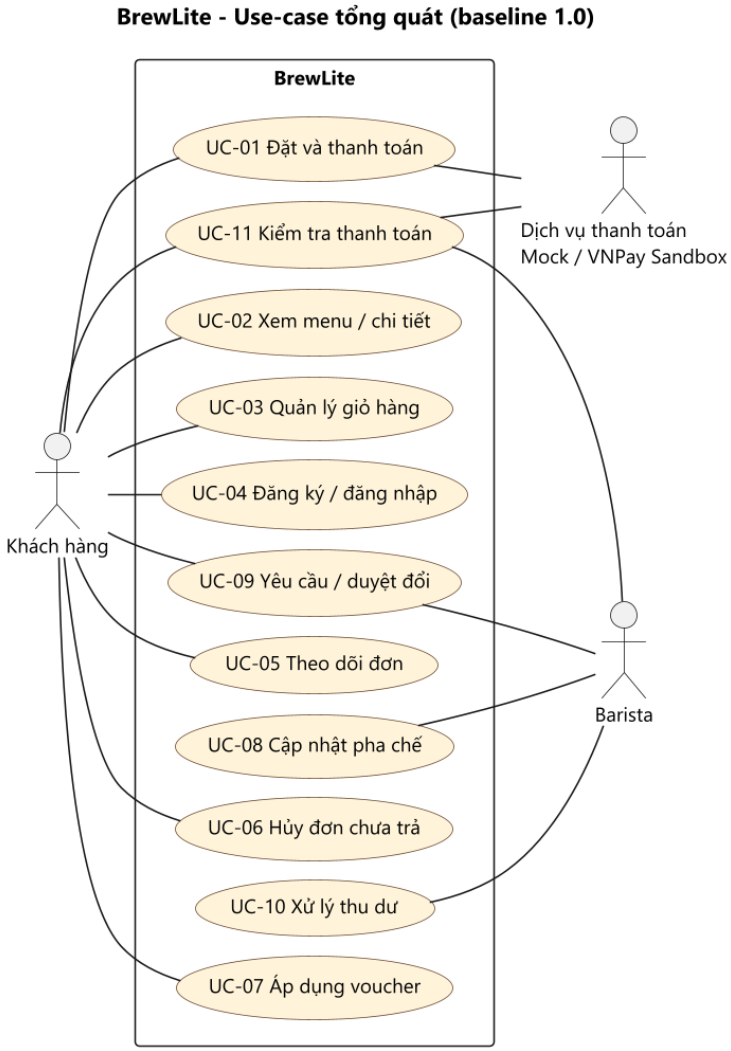
Hệ thống hướng tới quán nhỏ/vừa, barista kiêm xử lý thanh toán. Quản lý là stakeholder chốt chính sách; các vai trò truy cập được tổ chức thành CUSTOMER và BARISTA. Tiền VND lưu bằng số nguyên; frontend không truy cập PostgreSQL hoặc tự quyết định khoản thu. Khách chọn món vào giỏ chưa giữ kho; chỉ tạo Order hợp lệ mới giữ nguyên liệu.

Các rủi ro chính là tranh chấp phần cuối, gửi lặp đơn/callback, tiền về muộn, thu dư, đơn bỏ thanh toán giữ hàng, giá thay đổi trong lúc chờ đổi và hồ sơ bị hai nhân viên xử lý cùng lúc. Khóa dòng/transaction, unique key, snapshot và version giải quyết các rủi ro khác nhau. Timeout không chứng minh giao dịch thất bại hoặc ngân hàng đã hoàn tiền.

Tác vụ nền xử lý đơn hết thời gian giữ món dựa trên deadline lưu ở database. VNPay được sử dụng trong môi trường Sandbox; khoản thu không áp dụng được quầy xử lý theo hồ sơ và chứng từ. Nhu cầu giao hàng được ghi nhận ở mức nghiệp vụ; phạm vi giao diện tập trung khách và quầy.

3.5 Sơ đồ use-case

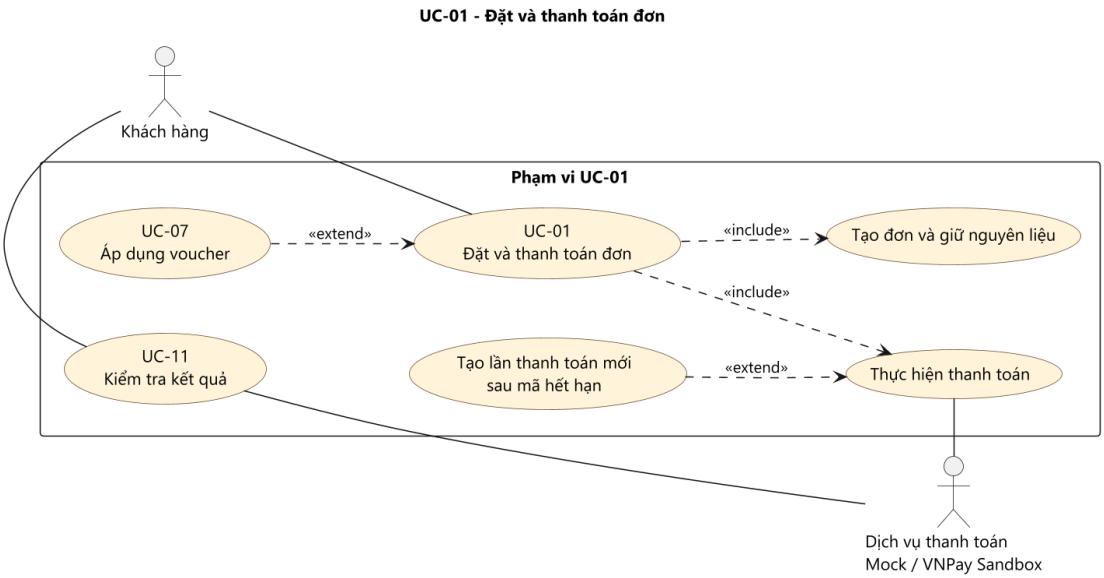
Hai sơ đồ tóm tắt phạm vi chức năng và tương tác trong luồng đặt, thanh toán đơn.



Hình 2: Use-case tổng quát, có nguồn PlantUML và bản SVG trong docs/diagrams

3.6 Yêu cầu nghiệp vụ đặt hàng, giữ kho và hoàn kho

Luồng đặt hàng được tổ chức thành ba thời điểm riêng biệt: khách chọn món, backend tạo đơn và hệ thống xác minh thanh toán. Việc thêm món vào giỏ chỉ lưu lựa chọn ở trình duyệt. Nguyên liệu được giữ khi backend chấp nhận tạo Order ở trạng thái PENDING; thanh toán thành công xác nhận quyền phục vụ đơn đã giữ, không trừ nguyên liệu thêm một lần nữa.



Hình 3: Phạm vi UC-01: đặt và thanh toán đơn

Mã quy tắc	**Nội dung và tiêu chí chấp nhận**
INV-01	Chỉ khách đã xác thực được tạo đơn. Backend kiểm tra sản phẩm, size, topping, số lượng và voucher; tự tính giá. Yêu cầu không hợp lệ không tạo đơn hoặc làm thay đổi kho.
INV-02	Khi tạo đơn, cộng gộp nhu cầu nguyên liệu của tất cả món và topping. Một nguyên liệu dùng chung chỉ có một tổng lượng cần giữ; vật tư không tăng theo size được tính riêng.
INV-03	Kiểm kho, trừ nguyên liệu, lưu Order/OrderItem, snapshot và kết quả idempotency thuộc cùng transaction. Một bước thất bại phải hoàn tác toàn bộ các thay đổi trong transaction.
INV-04	Nhiều khách tranh phần nguyên liệu cuối: tổng lượng giữ thành công không vượt tồn thực tế. Cùng khách, cùng key và cùng nội dung phải trả lại đơn cũ, không trừ kho lần hai.
INV-05	Khách chỉ tự hủy đơn PENDING hoặc PAYMENT_FAILED chưa sử dụng credit chuyển từ đơn cũ; phải sở hữu đơn. Giao dịch còn cần xác minh được kiểm tra trước khi cho hủy. Đơn PAID không đi qua API hủy thông thường.
INV-06	Hủy hợp lệ hoàn theo lượng đã lưu tại thời điểm đặt, không theo Recipe hiện tại. Hủy đồng thời hoặc gọi hoàn lại không được cộng kho hai lần; thiếu bằng chứng snapshot phải chuyển quầy kiểm tra.
INV-07	Đơn chưa thanh toán có thời hạn giữ món mặc định 30 phút. Hết hạn, backend chặn thanh toán mới và tác vụ nền đóng đơn đủ điều kiện, hoàn snapshot; giao diện thông báo khách đặt lại.

Mã quy tắc	Nội dung và tiêu chí chấp nhận
INV-08	Thời hạn QR không vượt thời hạn giữ món. QR hết hạn trước deadline của đơn được cấp lại; việc cấp QR không tự gia hạn giữ nguyên liệu. Tiền về muộn được ghi nhận vào quy trình xử lý khoản thu không áp dụng.
INV-09	Đơn đã có Payment, bằng chứng đã nhận tiền hoặc credit chuyển từ đơn cũ được bảo toàn để quầy xử lý; không tự đóng như đơn bỏ thanh toán thông thường. Khi duyệt đổi món, hoàn lượng cũ và giữ lượng mới phải cùng thành công hoặc cùng rollback.

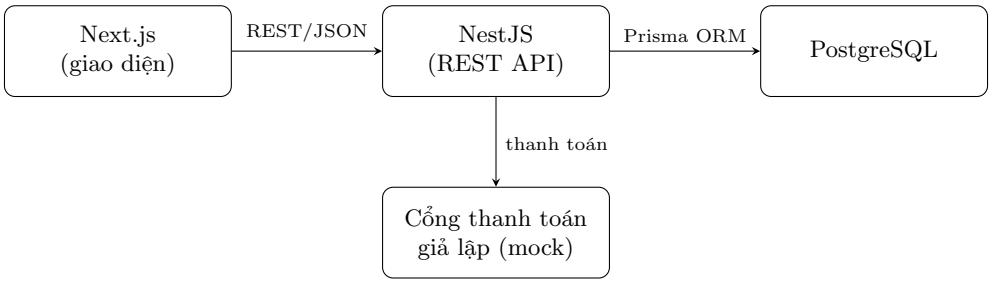
Bảng 16: Quy tắc và tiêu chí chấp nhận của luồng quản lý nguyên liệu

Ví dụ, khách chọn hai ly M có topping trong giỏ chưa làm kho giảm. Sau khi gửi đặt hàng thành công, lượng nguyên liệu đã giữ được ghi cùng mã đơn. Nếu khách chưa thanh toán và hủy hợp lệ, lượng này được cộng lại. Nếu khách trả tiền thành công, đơn chuyển PAID và quầy tiếp nhận; kho không bị trừ lần nữa khi chuyển trạng thái.

4 Thiết kế hệ thống

4.1 Kiến trúc tổng thể

Frontend Next.js gọi REST API của backend NestJS bằng JSON. Backend dùng Prisma để làm việc với PostgreSQL và tích hợp cổng thanh toán giả lập (mock payment gateway) dưới dạng một service riêng trong backend.



Hình 4: Kiến trúc tổng thể của hệ thống

4.2 Mô hình dữ liệu

Quan hệ: một User có nhiều Order; một Order có nhiều OrderItem và Payment; OrderItem tham chiếu Product. Ingredient và Recipe biểu diễn nguyên liệu và định lượng theo món. OrderInventoryReservation lưu từng dòng nguyên liệu đã giữ của từng đơn, tách biệt công thức hiện tại. OrderCreateRequest lưu khóa chống tạo đơn trùng; VnpayAttempt lưu từng lần thanh toán VNPay; OrderExchange lưu yêu cầu đổi món; PaymentRefundCase lưu hồ sơ xử lý khoản thu dư. LoyaltyTransaction là sổ giao dịch điểm thưởng. Phần giải thích bên dưới trình bày mục đích và ví dụ của các bảng bổ sung.

Thực thể	Thuộc tính
Product	id, name, price, imageUrl; tồn kho được suy ra từ nguyên liệu và công thức
User	id, email (duy nhất), passwordHash, loyaltyPoints
Voucher	id, code (duy nhất), discountPercent, expiresAt, active
Order	id, userId, status, total, discount, voucherId, createdAt
OrderItem	id, orderId, productId, size, toppings, qty, lineTotal
Payment	id, orderId, idempotencyKey (duy nhất), amount, method, createdAt

Bảng 17: Các thực thể của hệ thống

4.2.1 Các thực thể phục vụ quản lý nguyên liệu

Product biểu diễn món bán cho khách; Ingredient biểu diễn tài nguyên thật trong kho. Một nguyên liệu có thể được dùng cho nhiều món, vì vậy không thể coi số phần của mỗi Product là một kho độc lập. Ví dụ cà phê sữa và Americano cùng dùng cà phê rang xay: bán một món sẽ làm giảm khả năng bán món còn lại.

Thành phần	Vai trò và lý do thiết kế
Ingredient	Lưu id, tên, đơn vị và stock bằng số nguyên theo đơn vị nhỏ nhất (g, ml, cái). Ràng buộc stock không âm bảo vệ dữ liệu ở database.
Recipe	Bảng liên kết Product–Ingredient. quantity là định lượng một ly size S; cặp productId/ingredientId duy nhất ngăn khai báo trùng thành phần của cùng công thức. quantity phải dương.
Recipe.scalesWithSize	Quyết định lượng có tăng theo size hay không. Cà phê, sữa có thể tăng; ly, nắp thường giữ một cái cho mỗi ly. Không suy ra điều này từ giá bán.
ToppingRecipe	Lưu toppingCode, ingredientId và quantity cho một phần topping. Một topping có thể dùng nhiều nguyên liệu; một nguyên liệu có thể vừa thuộc công thức món vừa thuộc topping. Bảng riêng tránh lặp toàn bộ topping cho mỗi món.
Order và OrderItem	Order lưu trạng thái và giá trị đơn; OrderItem lưu món, size, topping và số lượng đã chọn. Những lựa chọn này là đầu vào để tính lượng cần giữ.
OrderInventoryReservation	Một dòng cho mỗi cặp orderId/ingredientId: quantity lưu chính tổng lượng đã trừ, releasedAt cho biết đã hoàn hay chưa. Đây là snapshot tài nguyên của đơn, không phải công thức sản phẩm thứ hai.
Order.reservationExpiresAt	Deadline giữ món được lưu bền vững, dùng chung cho worker, API thanh toán và giao diện. Không phụ thuộc thời điểm khách tải lại trang.
Order.reservationExpiredAt	Ghi thời điểm hệ thống đóng do hết hạn; phân biệt với khách chủ động hủy hoặc đơn cũ được thay thế khi đổi món dù đều dùng trạng thái CANCELLED.

Thành phần	Vai trò và lý do thiết kế
OrderCreateRequest	Lưu khách, idempotency key, dấu vân tay nội dung và Order kết quả. Ngăn retry tạo thêm đơn và giữ thêm nguyên liệu.

Bảng 18: Dữ liệu và trách nhiệm của các bảng trong luồng giữ/hoàn kho

Một Order có nhiều snapshot nguyên liệu; một Ingredient xuất hiện trong snapshot của nhiều Order. Unique(orderId, ingredientId) thể hiện việc cộng gộp trước khi lưu. Quan hệ snapshot tham chiếu nguyên liệu dùng quy tắc Restrict khi xóa, giữ bằng chứng lịch sử ngay cả khi công thức Recipe đã được thay đổi hoặc xóa. OrderItem lưu lựa chọn mua, còn snapshot lưu tài nguyên đã tiêu hao để giữ chỗ: hai loại dữ liệu phục vụ hai mục đích khác nhau.

```
model Payment {
  id Int @id @default(autoincrement())
  orderId Int
  idempotencyKey String @unique
  amount Int
  method PaymentMethod
  createdAt DateTime @default(now())
}
```

4.3 Thiết kế API

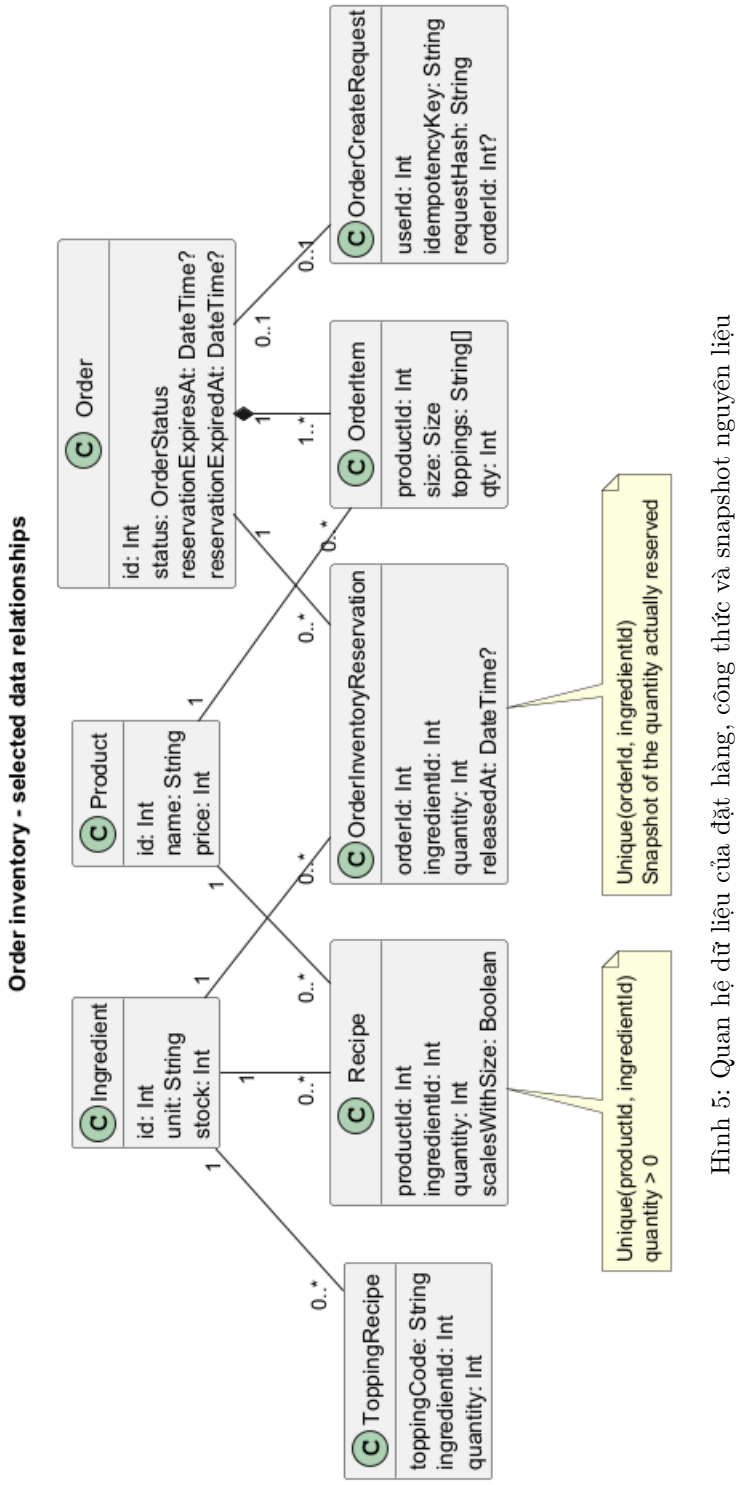
Phương thức	Đường dẫn	Chức năng	Đăng nhập
GET	`/products`	Danh sách sản phẩm	Không
GET	`/products/:id`	Chi tiết sản phẩm, kèm bảng size và topping	Không
POST	`/auth/register`	Đăng ký, trả JWT	Không
POST	`/auth/login`	Đăng nhập, trả JWT	Không
GET	`/auth/me`	Thông tin người dùng hiện tại	Có
POST	`/promotions/validate`	Kiểm tra mã giảm giá	Không
POST	`/orders`	Tạo đơn PENDING (kèm `voucherCode` tùy chọn)	Có
GET	`/orders/me`	Lịch sử đơn của người dùng	Có
GET	`/orders/:id`	Chi tiết một đơn	Có
POST	`/orders/:id/cancel`	Hủy đơn	Có
POST	`/payments`	Thanh toán, bắt buộc header `Idempotency-Key`	Có

Bảng 19: Các API của backend

4.3.1 Thiết kế xác thực và phân quyền

Hệ thống tách ba câu hỏi: người gọi là ai (JWT), có vai trò nào (Role), và có quyền trên đối tượng cụ thể không (chủ đơn). User lưu email duy nhất, passwordHash, role và điểm; role mặc định CUSTOMER. API đăng ký công khai không nhận quyền BARISTA từ dữ liệu khách gửi. Tài khoản quầy được cấp qua quy trình quản trị dữ liệu.

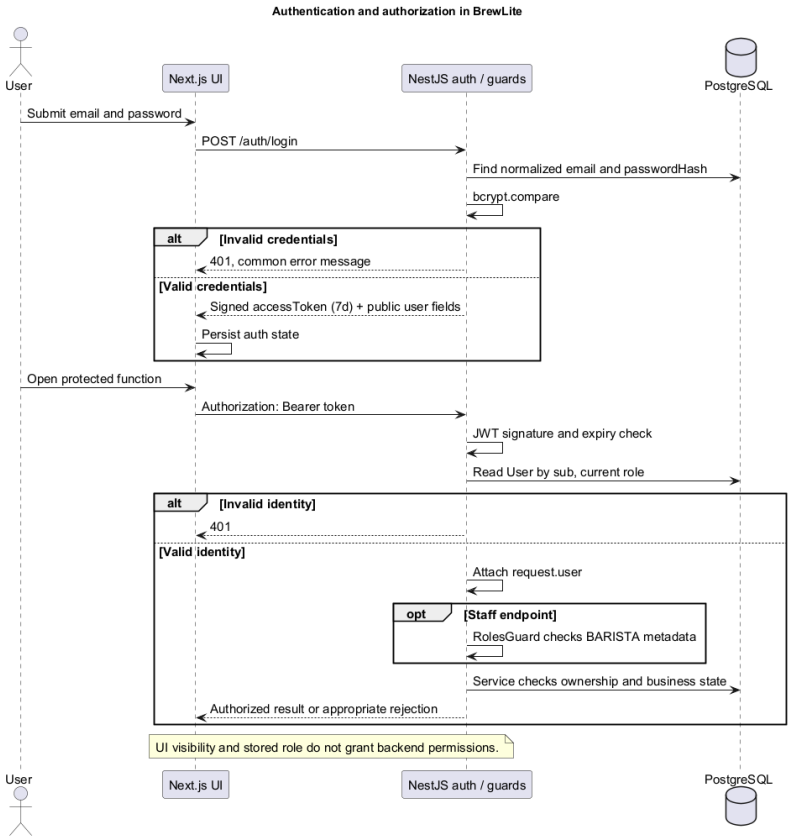
Thiếu token, token không hợp lệ/hết hạn hoặc người dùng không còn tồn tại bị từ chối xác thực (401). Tài khoản đã xác thực nhưng không có vai trò staff bị từ chối quyền (403). Với API đọc/hủy đơn cá nhân, đơn không thuộc người gọi được trả như không tìm thấy (404). Các mã lỗi có phạm vi khác nhau; việc ẩn nút quầy trên frontend không thay thế kiểm tra backend.



Hình 5: Quan hệ dữ liệu của đặt hàng, công thức và snapshot nguyên liệu

Nhóm thao tác	Điều kiện backend
Menu, chi tiết; đăng ký/đăng nhập	Được truy cập không cần JWT; input đăng ký/đăng nhập vẫn qua DTO validation.
Thông tin tài khoản hiện tại	JWT hợp lệ; danh tính lấy từ token đã xác minh và User trong database.
Xem/tạo/hủy/thanh toán đơn cá nhân	JWT hợp lệ và điều kiện nghiệp vụ; khi thao tác trên đơn có sẵn phải kiểm userId là chủ đơn.
Hàng đợi, cập nhật trạng thái và quyết định của quầy	JWT hợp lệ, role BARISTA và luật trạng thái/version/transaction tương ứng.
Khách xem/rút yêu cầu đổi	JWT và chủ Order nguồn; không có quyền tự duyệt hoặc tự xác nhận hoàn khoản thu dư.
IPN/Return từ VNPay	Dữ liệu giao thức được kiểm chữ ký và tham chiếu, không dùng JWT của khách làm chứng thực callback cổng.

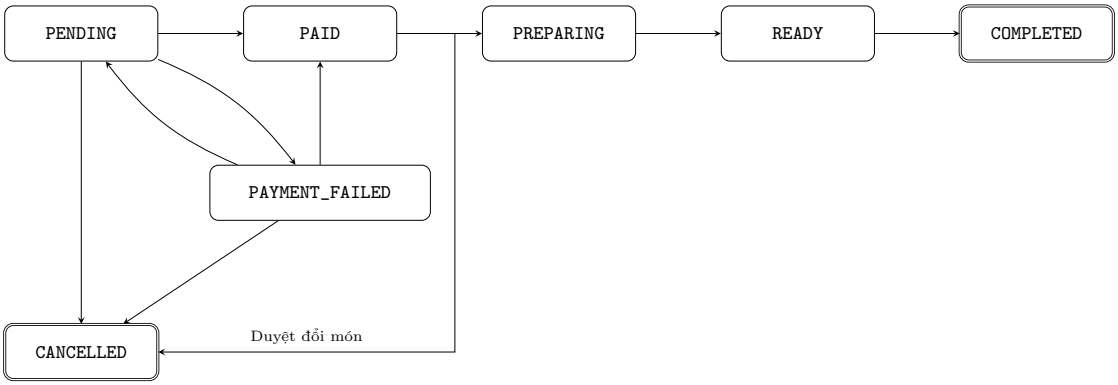
Bảng 20: Điều kiện truy cập theo danh tính, vai trò và chủ thể dữ liệu



Hình 6: Đăng nhập cấp token và chuỗi kiểm danh tính, vai trò, chủ đơn ở backend

4.4 Máy trạng thái đơn hàng (Order State Machine)

Mỗi đơn hàng tại một thời điểm chỉ ở đúng một trạng thái và chỉ được đổi theo các chuyển tiếp hợp lệ. Mọi chuyển tiếp không có trong sơ đồ bị hàm `assertTransition` chặn lại bằng lỗi. COMPLETED và CANCELLED là trạng thái kết thúc.



Hình 7: Sơ đồ máy trạng thái của đơn hàng

Từ	Sang	Sự kiện
PENDING	PAID	Thanh toán thành công
PENDING	PAYMENT_FAILED	Thanh toán lỗi
PENDING	CANCELLED	Khách hủy đơn
PAYMENT_FAILED	PENDING hoặc PAID	Thử thanh toán lại
PAYMENT_FAILED	CANCELLED	Khách hủy đơn
PAID	CANCELLED	Chỉ khi yêu cầu đổi món được duyệt, chuyển giá trị sang đơn mới
PAID	PREPARING	Barista nhận pha chế
PREPARING	READY	Pha xong
READY	COMPLETED	Giao cho khách

Bảng 21: Các chuyển trạng thái hợp lệ

Khách không được tự hủy đơn PAID. Chuyển PAID sang CANCELLED chỉ được thực hiện trong nghiệp vụ đổi món đã được barista duyệt, đồng thời chuyển giá trị đã trả sang đơn thay thế. Máy trạng thái chỉ kiểm tra chuyển tiếp; service phải kiểm tra quyền, điều kiện và giao dịch nguyên tử. Giao diện barista xử lý PREPARING, READY và COMPLETED. Khi đã pha chế, không áp dụng luồng đổi món này.

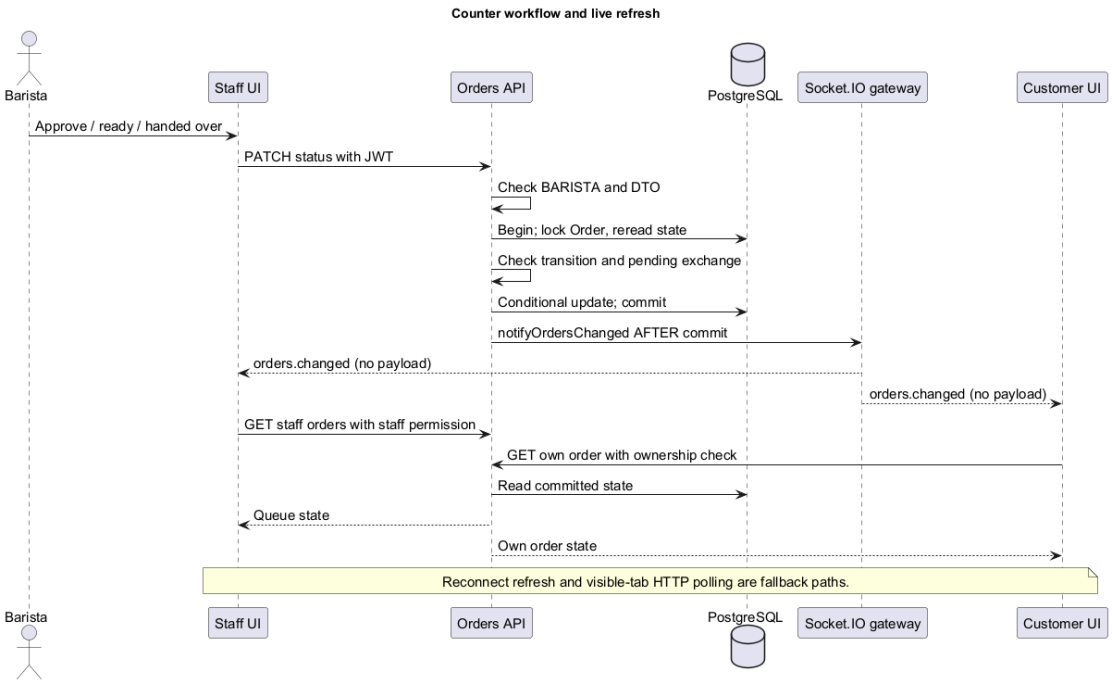
4.4.1 Vận hành quầy và đồng bộ trạng thái

Barista xử lý theo chuỗi PAID → PREPARING → READY → COMPLETED trên bảng bốn cột: chờ duyệt, đang pha, sẵn sàng và hoàn tất. Đơn PENDING/PAYMENT_FAILED/CANCELLED không vào hàng đợi pha chế. Thao tác quầy chỉ đổi trạng thái phục vụ, không tạo Payment, giữ nguyên liệu hoặc cộng điểm lần nữa. Quyền BARISTA áp dụng ở backend như mục 4.3.1.

findForStaff lấy mọi đơn đang xử lý theo createdAt/id tăng dần và tối đa 20 đơn hoàn tất gần nhất. Hai nhóm được đọc trong transaction RepeatableRead để tránh một đơn vừa chuyển

COMPLETED bị mất hoặc lặp do đọc ở hai thời điểm khác nhau. Giới hạn 20 chỉ áp dụng nhóm hoàn tất, không cắt mất đơn đang chờ làm.

Đơn có yêu cầu đổi REQUESTED phải được duyệt/từ chối/rút trước khi bắt đầu pha. Khi quầy cập nhật thành công, backend commit rồi phát sự kiện orders.changed. Sự kiện chỉ báo có thay đổi, không mang thông tin đơn/tài khoản; client gọi lại API có xác thực và quyền đọc tương ứng. WebSocket là tín hiệu đồng bộ, còn database/API là nguồn trạng thái thật.



Hình 8: Cập nhật trạng thái tại quầy, commit rồi thông báo để mỗi client đọc lại API được phép

4.5 Quy tắc tính giá, giảm giá và điểm thưởng

- **Giá một ly** = giá menu (giá size S) + phụ thu size + phụ thu topping. Phụ thu size: S cộng 0đ, M cộng 5.000đ, L cộng 10.000đ. Mỗi topping (Trân châu, Kem) cộng 5.000đ. Ví dụ: Cappuccino 45.000đ, size L, thêm Trân châu là 45.000 + 10.000 + 5.000 = 60.000đ.
- **Mã giảm giá:** giảm theo phần trăm, làm tròn xuống đến đồng, không giảm quá tổng tiền; mã có hạn dùng và có thể bị tắt. Mã mẫu BREW10 giảm 10%.
- **Điểm thưởng:** khách được cộng 1 điểm cho mỗi 10.000đ đã thanh toán (tính trên số tiền sau giảm), cộng khi đơn chuyển sang PAID.

Các mức phụ thu, mức giảm và tỷ lệ điểm là chính sách cấu hình của cửa hàng.

4.5.1 Mã giảm giá và giá trị đơn đã chốt

Voucher lưu code duy nhất, discountPercent, expiresAt và active; Order liên kết voucherId đồng thời lưu discount và total. Liên kết chỉ ra mã đã áp dụng, còn hai số tiền trên Order giữ giá trị giao dịch đã tính, không tính lại theo cấu hình voucher khi khách thanh toán sau đó.

Backend chuẩn hóa mã bằng trim/uppercase, kiểm tồn tại, active và expiresAt. Endpoint validate phục vụ xem trước; lúc tạo đơn vẫn kiểm lại trong transaction trước khi reserve nguyên liệu. Giá frontend hiển thị không thay thế phép tính backend.

$$discount = \min\left(subtotal, \left\lfloor subtotal \times \frac{p}{100} \right\rfloor\right), \qquad total = subtotal - discount.$$

Trong hàm tính, p được giới hạn trong khoảng 0–100. Ví dụ 120.000đ áp BREW10 giảm 12.000đ, còn 108.000đ. Mã không hợp lệ làm yêu cầu tạo đơn bị từ chối trước khi thay đổi kho.

4.5.2 Số dư điểm và sổ giao dịch điểm

User.loyaltyPoints là số dư hiện tại để đọc nhanh. LoyaltyTransaction lưu userId, orderId, loại, points, balanceAfter và thời gian để giải thích mỗi lần biến động. Unique(orderId, type) ngăn ghi hai dòng EARN cho cùng Order; balanceAfter ghi số dư sau lần cộng đó.

Chính sách hiện tại là $points = \max(0, \lfloor amount/10.000 \rfloor)$ theo khoản thanh toán mới được áp dụng. Đơn sau giảm 108.000đ được 10 điểm; dưới 10.000đ không tạo dòng EARN. Khi đổi món, tiền chuyển từ đơn cũ không cộng lại điểm; phần thu thêm được tính tại đơn thay thế. Điểm được làm tròn theo mỗi lần áp dụng, không gom phần tiền lẻ giữa các đơn.

4.6 VNPay Sandbox và xác minh kết quả

Tích hợp VNPay sử dụng ba thành phần: frontend hiển thị QR/liên kết và trạng thái; NestJS tạo yêu cầu, kiểm chứng kết quả và ghi sổ; VNPay Sandbox xử lý phiên thanh toán thử. QR chứa URL mở trang VNPay, không phải QR chuyển khoản trực tiếp của ứng dụng ngân hàng. Chữ ký và khóa merchant chỉ được xử lý ở backend. Cách phân biệt Return/IPN và quy tắc tham số được đối chiếu với tài liệu VNPay [1, 2].

4.6.1 Phân biệt đơn hàng, lần thanh toán và khoản áp dụng

Thực thể	Trách nhiệm
Order	Nghĩa vụ phục vụ và tiền còn cần thu: total trừ creditApplied. Một đơn có thể thử thanh toán nhiều lần.
VnpayAttempt	Mỗi lần giao dịch có txnRef duy nhất, số tiền, URL, hạn, mã giao dịch nhà cung cấp và trạng thái riêng; giữ lịch sử cả lần thất bại/chưa rõ.
PaymentStartRequest	Ánh xạ cặp khách/key tới attempt để gửi lại yêu cầu không tạo thêm lần thanh toán. Key mới vẫn có thể tái dùng mã đang hiệu lực.
Payment	Khoản đã xác minh và áp dụng vào Order; không tạo cho mọi attempt hoặc tiền thu nhưng không thể phục vụ. Liên kết attempt duy nhất chống ghi nhận lại cùng lần.
PaymentAudit	Nhật ký tạo mã, tra cứu, nhận kết quả và thao tác hỗ trợ; phục vụ truy vết theo attempt, actor và thời gian.
PaymentRefundCase	Hồ sơ khoản đã thu nhưng không áp dụng được (thu trùng, đơn không còn hiệu lực); một hồ sơ theo attempt, quầy xử lý bằng chứng từ.

Bảng 22: Phân tách dữ liệu thanh toán để bảo toàn lịch sử và tránh ghi nhận trùng

Ví dụ đơn 75.000đ đã chuyển credit 45.000đ chỉ cần thu thêm 30.000đ. Backend ký đúng phần còn thu; frontend không tự quyết định số tiền. Nếu lần A hết hạn rồi cấp B và cả hai cùng thu thành công, chỉ một khoản được áp dụng cho đơn; khoản còn lại có attempt và hồ sơ xử lý riêng, không tạo hai đơn pha chế.

4.6.2 Luồng và các nhánh kết quả

Backend kiểm chủ đơn, PENDING/PAYMENT_FAILED, deadline và snapshot chưa hoàn trước khi cấp mã. Lần mới có txnRef riêng; amount lưu VND nguyên, trường vnp_Amount gửi sang cổng nhân 100 theo giao thức. Các trường được chuẩn hóa và ký HMAC-SHA512. Hạn QR bị giới hạn bởi deadline giữ món đã trình bày ở mục 4.7.1.

Return xác minh chữ ký, merchant, tham chiếu và số tiền rồi trả HTTP 303 về trang Order; không chuyển PAID. IPN là thông báo server-to-server; QueryDr là truy vấn backend chủ động khi cần kiểm tra. Cả hai kết quả đã xác minh được quy về apply để cùng kiểm soát ghi nhận. Mất Return hoặc khách đóng tab không xóa kết quả đã commit.

Tình huống	**Xử lý trong hệ thống**
Thành công, đơn còn đủ điều kiện	Ghi Payment, PAID và điểm trên tiền mới trong cùng transaction; không giữ kho thêm.
Thất bại đã xác minh	Ghi FAILED/EXPIRED theo kết quả; đơn còn hạn có thể bắt đầu lần khác.
Timeout hoặc kết quả chưa rõ	Giữ lịch sử PENDING/REVIEW, không tự suy ra thất bại. QueryDr chạy ngoài transaction; việc cấp mã mới tuân điều kiện hạn mã/đơn và bằng chứng tiền đã thu.
QR hết hạn, đơn còn hạn	Thử tra cứu rồi có thể cấp lần mới dù tra cứu chưa kết luận; giữ attempt cũ để nhận kết quả muộn.
IPN lặp hoặc đến ngược thứ tự	Kết quả đã ghi không được ghi lại/cộng điểm lại; thông báo thất bại đến sau không hạ PAID.
Thu trùng hoặc tiền về sau khi đơn không còn hiệu lực	Ghi REVIEW và hồ sơ khoản thu không áp dụng; không tự mở lại đơn/pha chế.

Bảng 23: Quy tắc xử lý kết quả VNPay và thanh toán lại

4.7 Giữ kho và hoàn theo snapshot

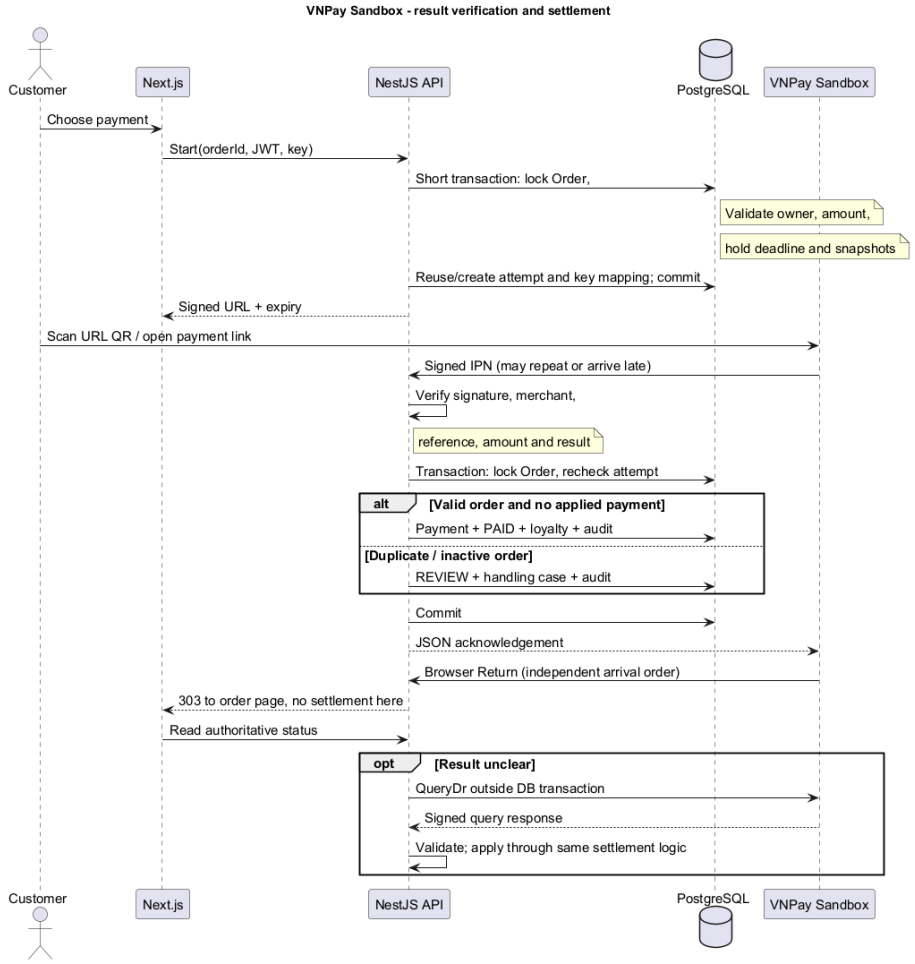
4.7.1 Định lượng theo size và topping

Với mỗi dòng món, backend lấy định lượng size S từ Recipe. Chính sách định lượng đang dùng có hệ số S = 100%, M = 125%, L = 150%; đây là cấu hình nghiệp vụ, độc lập phụ thu giá size. Thành phần có scalesWithSize=true được nhân hệ số rồi làm tròn lên theo từng ly. Thành phần có scalesWithSize=false giữ nguyên lượng. Sau đó cộng định lượng topping của một ly và nhân số ly; cuối cùng gộp tất cả dòng theo ingredientId.

Gọi q_{ig} là lượng cơ bản nguyên liệu g của món i, r_s là hệ số size, n_i là số ly và t_{ig} là tổng lượng nguyên liệu g từ topping của một ly. Khi thành phần tăng theo size:

$$D_g = \sum_i n_i \left(\lceil q_{ig} r_s \rceil + t_{ig} \right).$$

Nếu không tăng theo size, thay $\lceil q_{ig} r_s \rceil$ bằng q_{ig}. Việc làm tròn trước khi nhân số ly bảo đảm mỗi ly đều có đủ lượng theo định mức nguyên.



Hình 9: Luồng VNPay: Return điều hướng, IPN/QueryDr xác minh và ghi nhận tại backend

Thành phần	**Một ly M**	**Hai ly M**
Cà phê cơ bản 18g, tăng theo size	$\lceil 18 \times 1{,}25 \rceil = 23$g	46g
Topping dùng thêm 5g cùng nguyên liệu	5g	10g
Ly nhựa, không tăng theo size	1 cái	2 cái
Tổng cà phê phải giữ	28g	56g

Bảng 24: Ví dụ minh họa cộng gộp định lượng, không phải công thức mặc định của topping Kem

Hàm ingredientDemand dùng chung cho kiểm kho khi đặt và tính availability trên menu. Với lựa chọn một ly, số ly có thể bán bằng giá trị nhỏ nhất của $\lfloor stock_g / D_g \rfloor$ trên các nguyên liệu cần dùng. Do topping có thể dùng chung nguyên liệu với món, phải gộp demand trước khi chia; tính riêng hai phần rồi lấy min có thể báo còn hàng sai. Availability ở giao diện là ảnh chụp tham khảo; backend luôn kiểm lại tại transaction tạo đơn hoặc duyệt đổi món.

4.7.2 Snapshot: vì sao không hoàn bằng công thức hiện tại?

Recipe mô tả định mức bán hiện hành, trong khi snapshot là bằng chứng lượng đã giữ của một đơn cụ thể. Ví dụ khi đặt, một ly giữ 20g cà phê. Sau đó quầy sửa Recipe thành 25g. Nếu hủy bằng công thức mới, hệ thống cộng 25g dù trước đó chỉ trừ 20g, tạo ra 5g không có thật. Snapshot giữ quantity=20 nên hoàn đúng 20g. Thay topping, hệ số size hay chuyển công thức sang nguyên liệu khác cũng không thay đổi snapshot đã lưu.

Snapshot được tạo từ chính Map demand đã dùng để trừ kho, trong cùng transaction với Order. Không đọc lại Recipe sau bước trừ để dựng snapshot, vì lần đọc khác có thể không đại diện lượng vừa giữ. releasedAt là dấu hoàn; quantity vẫn giữ nguyên để tra cứu lịch sử. Một đơn có nhiều dòng món cùng dùng cà phê sẽ có một snapshot cà phê tổng, vì khi hủy toàn đơn chỉ cần trả đúng tổng lượng đã giữ.

4.7.3 Hủy chủ động và hết thời gian giữ món

Hủy chủ động kiểm tra chủ đơn, trạng thái và điều kiện thanh toán. Khách chỉ tự hủy PENDING/PAYMENT_FAILED không có credit; backend khóa Order, kiểm tra các attempt cần xác minh, cập nhật trạng thái có điều kiện rồi release snapshot trong cùng transaction. Nếu bước hoàn bị lỗi thì trạng thái hủy cũng rollback. PAID và các trạng thái pha chế không dùng API hủy thông thường; đổi món đã trả tiền đi qua yêu cầu và quyết định của barista.

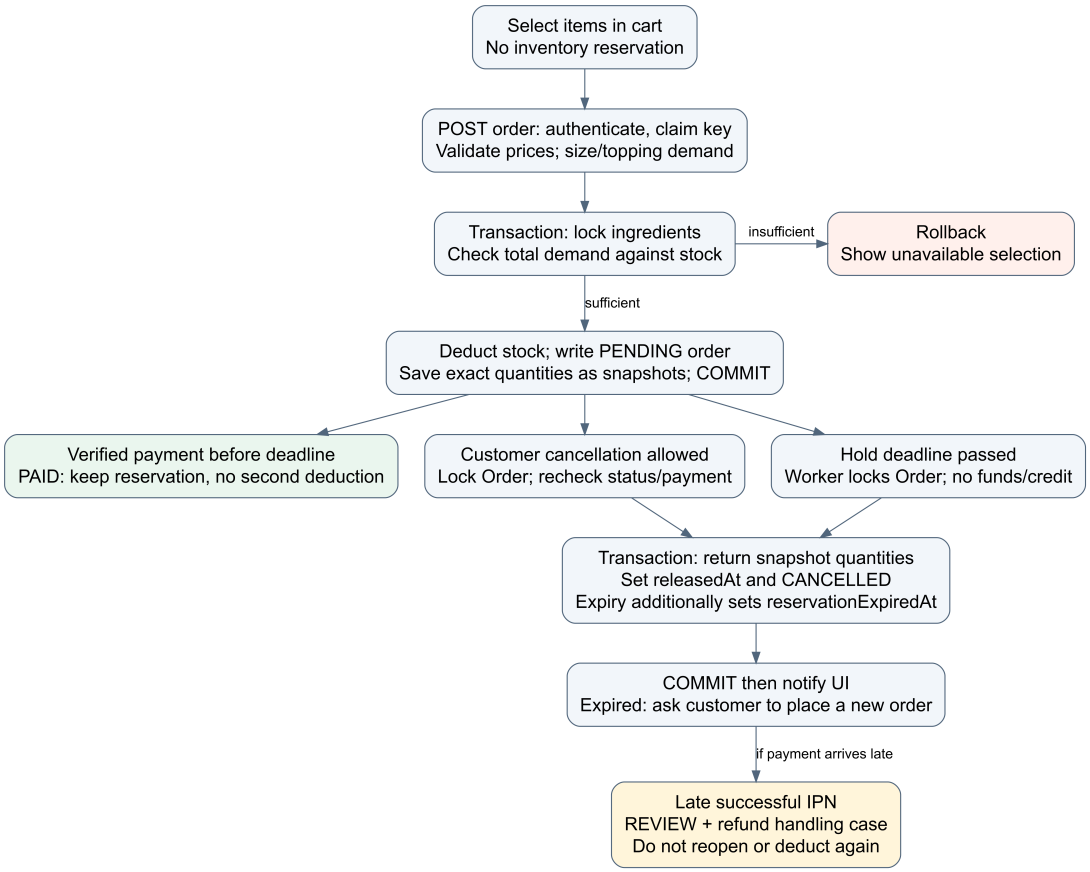
Với đơn bỏ thanh toán, reservationExpiresAt mặc định cách lúc tạo 30 phút. ReservationExpiryService quét khi khởi động và mỗi 30 giây, tối đa 100 đơn/lượt. Chỉ đóng đơn chưa trả tiền, chưa có bằng chứng đã thu và chưa sử dụng credit. Khi đủ điều kiện, worker hoàn snapshot và ghi CANCELLED cùng reservationExpiredAt; thông báo được phát sau commit. Giao diện hiển thị "Đã hết thời gian đặt món, vui lòng đặt lại." và đưa khách về menu để tạo yêu cầu mua mới.

Deadline của đơn và deadline QR là hai giá trị khác nhau. Ví dụ đơn giữ món từ 10:00 đến 10:30; QR đầu hết 10:10 thì vẫn cấp QR mới trong thời gian còn giữ. Nếu tạo QR lúc 10:25, QR không được kéo dài quá 10:30. Backend kiểm deadline tại start payment và lúc áp dụng IPN, nên timer trình duyệt không quyết định quyền giữ kho.

Nếu tiền về muộn sau deadline, hệ thống giữ bằng chứng giao dịch, ghi REVIEW và mở hồ sơ xử lý khoản thu không áp dụng; không tự mở lại đơn, trừ kho lại hoặc gửi pha chế. Nếu worker chưa đóng đơn, callback xử lý việc đóng/hoàn trong transaction thích hợp. Các đơn đã chuyển credit hoặc có bằng chứng thu tiền được bảo toàn để quầy xử lý, tránh tự xóa giá trị khách đã trả.

4.8 Đổi món, chuyển tiền và thanh toán chênh lệch

Đổi món là quy trình thay thế một đơn đã thanh toán trước khi pha chế, không phải khách tự hủy để nhận tiền. Khách mở liên kết hỗ trợ trên Order PAID, chọn lại món/size/topping qua menu và giỏ đổi riêng; backend báo giá, khách gửi lý do và quầy quyết định. Mở menu hoặc nhận báo giá không tự hoàn kho hay đóng đơn cũ.



Hình 10: Luồng tạo đơn, giữ kho, thanh toán, hủy và hết thời hạn giữ món

4.8.1 Dữ liệu yêu cầu và báo giá

OrderExchange lưu sourceOrderId, replacementOrderId, items, reason, trạng thái, quyết định và version. Liên kết hai Order giữ được lịch sử trước/sau thay vì sửa trực tiếp các dòng món của đơn đã thu tiền. Unique(sourceOrderId, key) cùng requestHash bảo vệ gửi lặp yêu cầu.

quotedLines/quotedTotal lưu báo giá đã nhận khi yêu cầu được tạo. Frontend gửi expectedQuote để backend đối chiếu fingerprint của báo giá; nếu giá/trạng thái đổi trước lúc gửi, khách phải xem lại. Fingerprint là dấu vân tay đối chiếu dữ liệu, không phải chữ ký cổng thanh toán. Khi đã tạo yêu cầu, bước duyệt dùng thành tiền snapshot, tránh tính lại theo giá menu mới trong lúc khách chờ.

Trạng thái	**Ý nghĩa và tác động**
REQUESTED	Chờ quầy; đơn nguồn vẫn PAID và giữ nguyên liệu. Chặn chuyển PREPARING cho tới khi xử lý yêu cầu.
APPROVED	Đã tạo đơn thay thế, chuyển credit và đóng đơn nguồn trong cùng transaction.
REJECTED	Quầy từ chối, lưu ghi chú; giữ nguyên món và tiền của đơn nguồn.
WITHDRAWN	Khách rút yêu cầu còn chờ bằng version; tiếp tục đơn nguồn.

Bảng 25: Vòng đời yêu cầu đổi món, độc lập vòng đời Order

4.8.2 Chuyển giá trị đã trả và thu chênh lệch

Giá trị có thể chuyển bằng tổng Payment đã áp dụng của đơn nguồn cộng creditApplied đã chuyển vào đơn đó; backend kiểm bằng total của đơn nguồn trước khi duyệt. Credit là giá trị gắn với đơn thay thế, không phải số dư ví dùng tùy ý.

Trường hợp	**Tiền cũ**	**Món mới**	**Thu thêm**
Đổi bằng giá	45.000đ	45.000đ	0đ
Đổi cao hơn	45.000đ	75.000đ	30.000đ
Đổi thấp hơn	45.000đ	40.000đ	Yêu cầu chọn lại

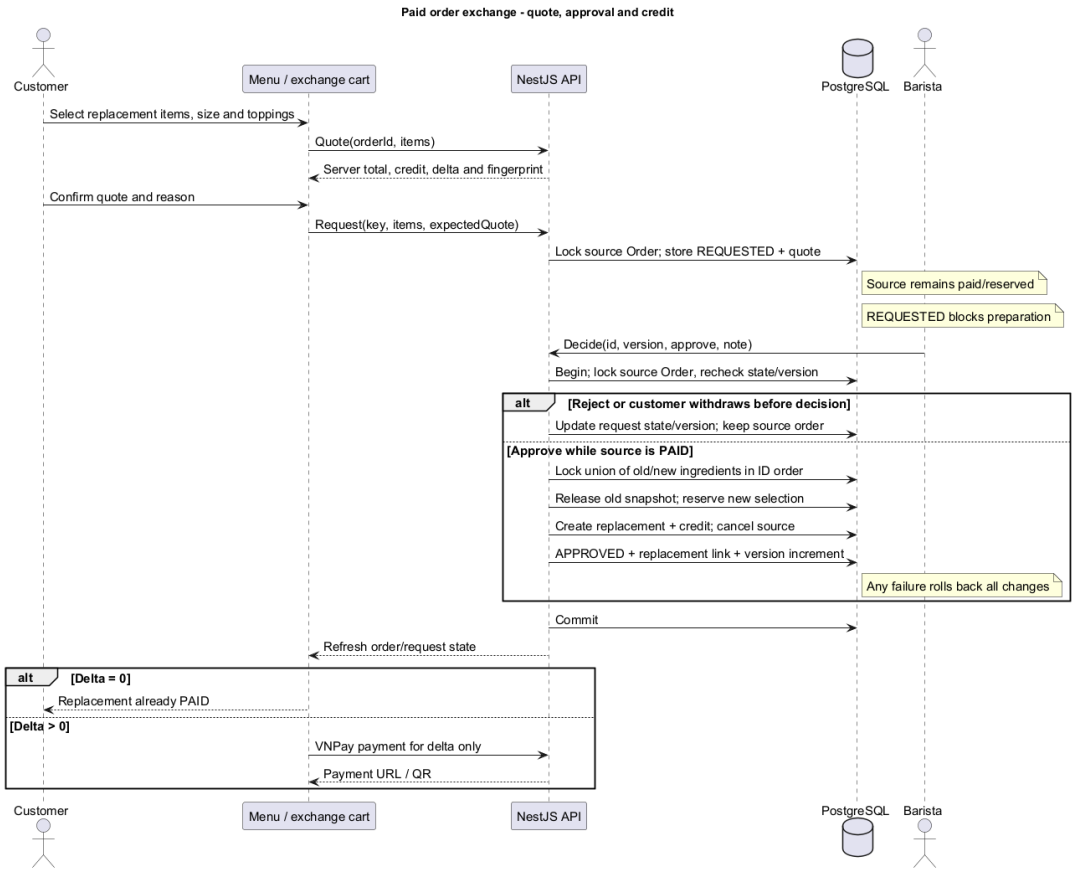
Bảng 26: Ví dụ credit và số tiền thu thêm khi đổi món

Order mới giữ total bằng giá trị món, creditApplied bằng tiền chuyển, amountDue bằng phần chênh lệch. Đổi bằng giá tạo Order PAID nhưng không tạo Payment giả hoặc cộng điểm trên tiền cũ lần nữa. Đổi cao hơn tạo Order PENDING; VNPay chỉ thu phần chênh lệch và điểm tính trên tiền mới. Quy tắc hiện tại không áp voucher mới trong yêu cầu đổi; phần thu dương qua tích hợp VNPay phải từ 5.000đ. Quầy chỉ bắt đầu pha khi đơn thay thế đã đủ tiền.

4.9 Version trong hồ sơ xử lý và khoản thu dư

OrderExchange và PaymentRefundCase dùng version để phát hiện cập nhật trên dữ liệu cũ. Hai nhân viên cùng đọc version=0: người ghi trước cập nhật với điều kiện id/version/trạng thái rồi tăng lên 1; người còn lại bị 409. Đây là optimistic concurrency cho hồ sơ; việc giữ nguyên liệu sử dụng khóa dòng trong transaction.

PaymentRefundCase liên kết duy nhất với attempt đã thu nhưng không áp dụng được. Quy trình OPEN, PROCESSING, COMPLETED có ghi chú, người xử lý, thời gian và chứng từ duy

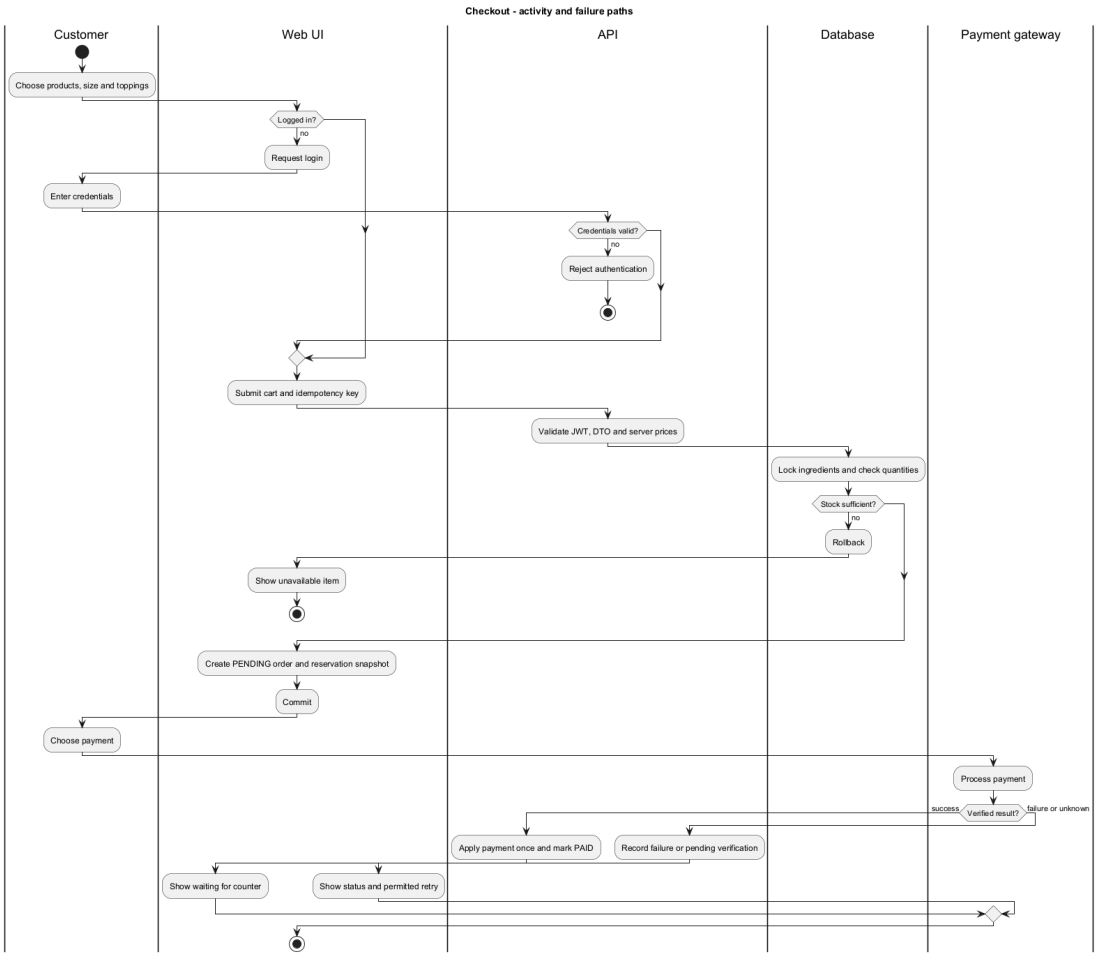


Hình 11: Khách gửi yêu cầu, quầy quyết định, transaction thay đơn và thanh toán chênh lệch

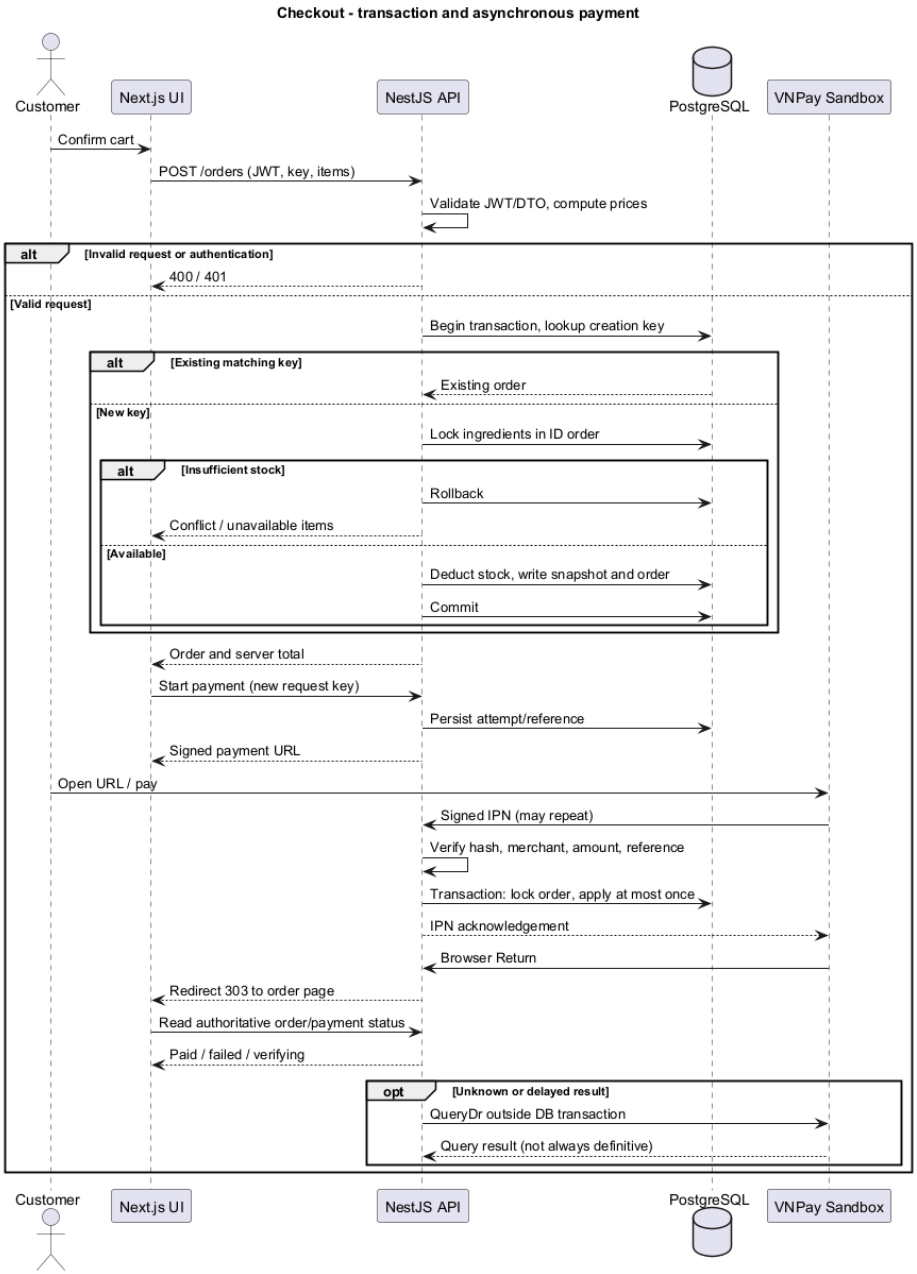
nhất. Nhân viên thực hiện chuyển trả bên ngoài rồi ghi nhận bằng chứng; backend theo dõi bằng chứng từ và nhật ký xử lý. Thu trùng khác với khách đổi ý sau trả tiền và không bị ép sử dụng khoản dư để mua món khác.

4.10 Mô hình hoạt động, tương tác và miền dữ liệu

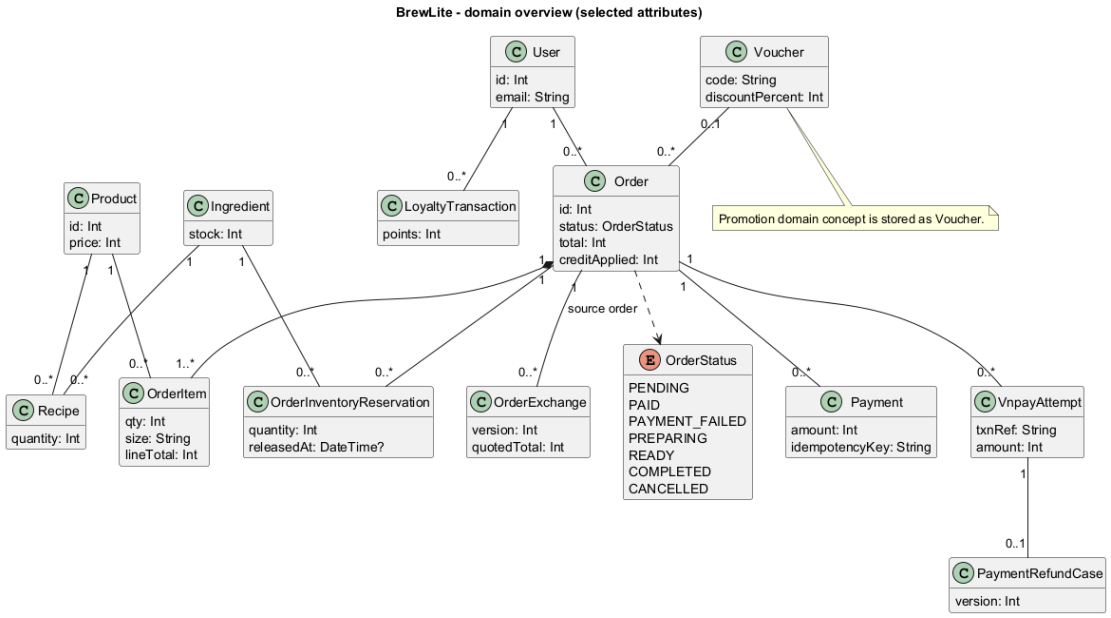
Ba hình dưới mô tả luồng đặt/thanh toán và mô hình miền ở mức chọn lọc, không thay thế kiểm thử hoặc chứng minh triển khai. Nguồn PlantUML và ảnh render được lưu cùng nhau tại `docs/diagrams/`. Voucher biểu diễn chính sách mã giảm giá; sổ LoyaltyTransaction lưu biến động điểm. Class diagram rút gọn thuộc tính, không phải bản sao đầy đủ schema.



Hình 12: ACT-01: hoạt động đặt hàng với swimlane và nhánh chưa đăng nhập, hết kho, thanh toán lỗi



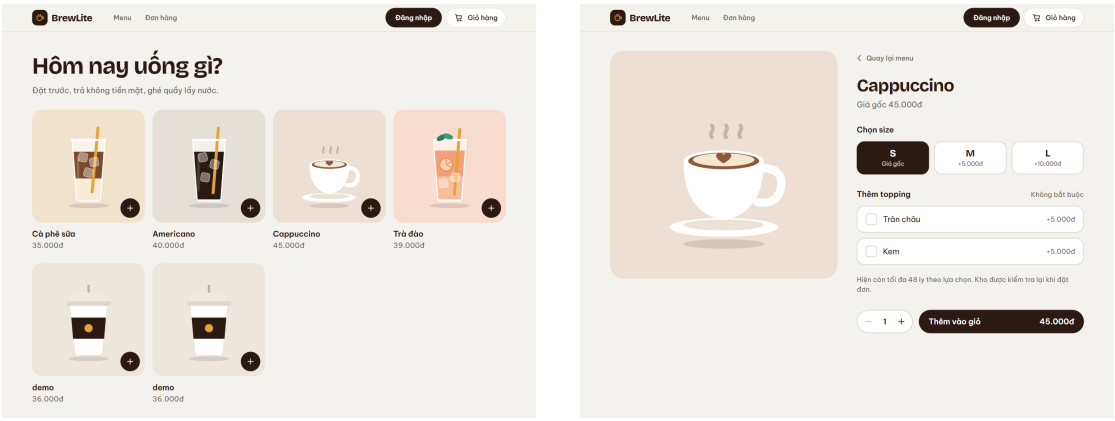
Hình 13: SEQ-01: tương tác UI, API, database và gateway; transaction và callback bất đồng bộ



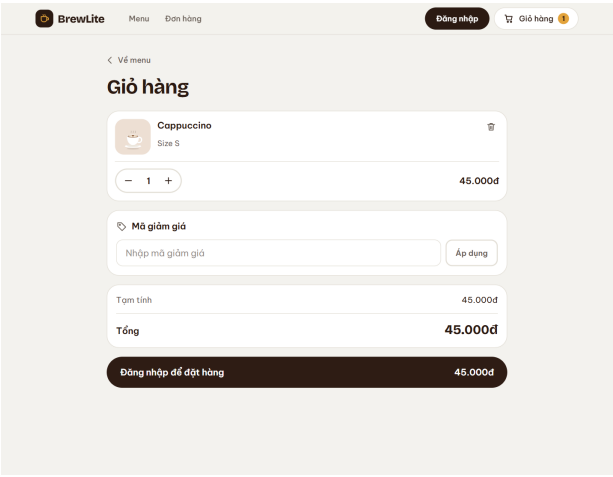
Hình 14: CLASS-01: các thực thể chính, quan hệ và bội số

4.11 Thiết kế giao diện

Menu ưu tiên tên món, giá và hình ảnh; trang chi tiết gom size, topping và số lượng trước thao tác thêm giỏ. Giỏ hàng cho sửa lựa chọn và xem tổng trước khi tạo đơn. Các màn hình thanh toán, đổi món và quầy được minh họa tại phần nghiệp vụ tương ứng.



Hình 15: Menu và chi tiết món trên môi trường local, dùng API danh mục



Hình 16: Giỏ hàng trước khi gửi đặt đơn; thao tác thêm món chỉ lưu ở trình duyệt

5 Hiện thực hệ thống

5.1 Cấu trúc thư mục

```
brewlite/
  docker-compose.yml
  README.md
  backend/ (NestJS)
    prisma/ schema, migrations, seed
    src/
      auth/ users/ products/ promotions/
      orders/ payments/ common/ prisma/
    test/ test end-to-end
  frontend/ (Next.js)
    src/
      app/ cac trang
      features/ auth, catalog, cart, orders, payments
      components/ lib/
```

Mỗi tính năng nằm trong thư mục riêng ở cả backend (module của NestJS) và frontend (thư mục `features`) để các thành viên ít đụng vào code của nhau.

5.2 Menu, chi tiết sản phẩm và giỏ hàng

5.2.1 Tải menu và trạng thái giao diện

MenuPage là Server Component gọi GET /products qua lớp API. ProductGrid hiển thị lưới responsive, ProductCard dẫn tới chi tiết theo id; danh sách rỗng có thông báo riêng, lỗi kết nối có nút tải lại. loading.tsx cung cấp khung chờ khi route tải dữ liệu. ProductImage dùng ảnh sản phẩm và minh họa dự phòng khi URL rỗng/ảnh lỗi, tránh đưa src rỗng vào thẻ img.

5.2.2 Tùy chọn và giá hiển thị

GET /products/:id cung cấp giá và danh mục phụ thu size/topping từ backend. ProductDetail giữ size, toppings, qty bằng state cục bộ, dùng calcUnitPrice để phản hồi ngay khi chọn. Ví dụ Cappuccino 45.000đ, size L thêm 10.000đ, một topping thêm 5.000đ thành 60.000đ. Availability theo tổ hợp hỗ trợ khóa lựa chọn thiếu nguyên liệu; đây là thông tin tại thời điểm đọc, không giữ hàng trong giỏ. Giá và lượng kho được backend kiểm lại khi tạo đơn như mục 3.6.

5.2.3 State giỏ hàng và lưu qua lần tải lại

Zustand lưu items, còn tổng tiền và badge được suy ra qua selector để không có nhiều bản sao tổng cần đồng bộ. Khóa dòng giỏ kết hợp productId, size và danh sách topping đã sort: cùng lựa chọn gộp số lượng, khác size/topping tách dòng. setQty cập nhật số lượng hoặc xóa khi về 0, giới hạn thao tác thông thường ở 99 ly/dòng.

persist lưu giỏ vào localStorage; useHydrated dùng snapshot server/client để tránh render số lượng lưu ở trình duyệt trước khi dữ liệu đó sẵn sàng. Đây là giỏ phía client theo origin, không phải một Order hoặc dữ liệu kho đã được giữ. Không tin unitPrice trong storage là giá thanh toán cuối.

5.2.4 Gửi đặt hàng và khôi phục khi mất phản hồi

CartView kiểm đăng nhập, chuyển tới login có next=/cart khi cần; khi đặt, chỉ gửi productId/-size/topping/qty và mã voucher. placingRef chặn gửi chồng trong một component. prepareCheckoutIntent lưu key/nội dung theo tài khoản trong sessionStorage trước khi gửi: cùng nội dung sau mất phản hồi dùng lại key, không âm thầm tạo đơn mới. Backend idempotency vẫn là lớp bảo đảm như phần tạo đơn.

Chỉ sau response xác nhận Order, client xóa giỏ, hoàn tất intent và chuyển tới trang đơn. Nếu lỗi hoặc chưa rõ kết quả, giữ ý định để kiểm tra/retry phù hợp. SessionStorage không chứa token của intent; token đăng nhập được lớp auth quản lý riêng. Việc tính giá frontend hỗ trợ trải nghiệm, còn transaction backend chốt giá và giữ kho.

frontend/src/features/cart/cart.store.ts · lines 28–30

```
28  function makeKey(item: NewCartItem): string {
29      return
    `${item.productId}|${item.size}|${[...item.toppings].sort().join(',')}`;
30  }
```

Hình 17: Khóa dòng giỏ được chuẩn hóa theo sản phẩm, size và topping

frontend/src/features/cart/checkout-intent.ts · lines 40–43

```
40      const key = makeKey();
41      // Nếu không lưu được thì không gửi request: tránh tự retry bằng key
    mới.
42      storage.setItem(storageKey, JSON.stringify({ key, body }));
43      return { key, body, items: normalizedItems, voucherCode:
    normalizedVoucher };
```

Hình 18: Lưu ý định đặt hàng trước khi gửi để tái sử dụng key khi cần

frontend/src/features/cart/CartView.tsx · lines 83–91

```
83              const order = await createOrder(intent.items,
    intent.voucherCode, intent.key);
84              if (useAuthStore.getState().token !== auth.token) {
85                  setError('Đơn đã được ghi nhận. Đăng nhập đúng tài khoản
    để xem lịch sử đơn.');
86                  return;
87              }
88              clearCart();
89              completeCheckoutIntent(auth.user.id, intent.key);
90              succeeded = true;
91              router.push(`/orders/${order.id}`);
```

Hình 19: Gửi lựa chọn và chỉ xóa giỏ sau khi backend xác nhận đơn

5.3 Xác thực

5.3.1 Đăng ký, đăng nhập và dữ liệu công khai

AuthService chuẩn hóa email bằng trim/lowercase. Đăng ký kiểm email, băm mật khẩu bằng bcrypt với cost 10 rồi tạo User CUSTOMER; đăng nhập dùng compare với hash đã lưu. Sai email hoặc mật khẩu trả cùng thông báo. UsersService dùng select các trường công khai khi tạo/đọc theo id; response không trả passwordHash.

AuthModule cấp access token thời hạn 7 ngày, payload nghiệp vụ gồm sub=id và email; thư viện quản lý trường thời gian. JWT được ký để kiểm chứng, không dùng như dữ liệu bí mật đã mã hóa. Secret lấy ở backend qua cấu hình môi trường. Frontend gửi token qua Authorization: Bearer khi gọi API được bảo vệ.

5.3.2 Chuỗi kiểm tra JWT, vai trò và chủ đơn

JwtAuthGuard dùng Passport strategy jwt để kiểm chữ ký và hạn. JwtStrategy.validate đọc lại User theo sub và trả id/email/role vào request.user; vì role lấy từ database ở mỗi lần validate, thay quyền có hiệu lực ở request xác thực tiếp theo mà không đợi token cũ hết hạn. CurrentUser chỉ trích request.user cho controller, không tự xác minh token lần nữa.

Route quầy khai báo UseGuards(JwtAuthGuard, RolesGuard) và Roles(BARISTA). RolesGuard đọc metadata của handler/class, đối chiếu role đã lấy từ database. Với route cá nhân, service còn kiểm userId của Order; có JWT đúng chưa đồng nghĩa được truy cập mọi mã đơn. Việc quyết định trạng thái, credit và version tiếp tục áp dụng sau khi qua lớp quyền.

5.3.3 Trạng thái đăng nhập ở giao diện

AuthForm gọi API, lưu accessToken/user vào Zustand persist và điều hướng về trang đang cần hoặc quầy theo role. Tham số next chỉ chấp nhận đường dẫn nội bộ để tránh điều hướng ra website tùy ý. Auth store hiện lưu qua localStorage; logout xóa trạng thái ở trình duyệt. Đây là thao tác kết thúc phiên phía client, không được hiểu là đã thu hồi mọi JWT đã phát hành. Dữ liệu role trên UI phục vụ hiển thị; backend vẫn quyết định quyền thật.

5.4 Tạo đơn và tích hợp thanh toán

`POST /orders` nhận danh sách món (mã sản phẩm, size, topping, số lượng), tự lấy giá từ cơ sở dữ liệu, tính lại tổng tiền và lưu đơn PENDING. Giá do trình duyệt gửi lên không được tin.

Cổng giả lập dùng để kiểm tra luồng Ví/Thẻ có kiểm soát; khi bật VNPay, endpoint mock bị chặn để không bỏ qua cổng. Luồng VNPay tách quá trình khách thao tác ở cổng khỏi transaction ghi nhận kết quả tại backend.

5.4.1 Hiện thực VNPay và bảo đảm ghi nhận một lần

VnpayGateway chịu trách nhiệm cấu hình, build URL và QueryDr. Các hàm canonical/hmac/readCallback tách xử lý chữ ký khỏi nghiệp vụ. VnpayService.start/replay kiểm yêu cầu bắt đầu; ipn/returnUrl phân biệt hai kênh; apply ghi nhận kết quả; check điều phối đối soát theo yêu cầu. Frontend PaymentQr/VnpayPanel chỉ hiển thị và gọi API.

5.4.1.1 Xác minh trước khi ghi. Callback phải có dạng dữ liệu hợp lệ; canonical loại trường hash, sắp khóa và encode một lần; HMAC so bằng timingSafeEqual sau kiểm độ dài/định dạng. Chữ ký đúng vẫn phải kiểm merchant, txnRef, amount và mã trạng thái. Nhóm không xem

backend/src/auth/auth.service.ts · lines 27–37

```
27      async register(dto: RegisterDto) {
28          const email = dto.email.trim().toLowerCase();
29
30          if (await this.users.findByEmail(email)) {
31              throw new ConflictException('Email đã được đăng ký');
32          }
33
34          const passwordHash = await hash(dto.password, 10);
35          const user = await this.users.create(email, passwordHash);
36          return this.buildResponse(user);
37      }
```

Hình 20: Đăng ký chuẩn hóa email và lưu hash thay vì mật khẩu gốc

backend/src/auth/jwt.strategy.ts · lines 17–24

```
17      async validate(payload: { sub: number; email: string }) {
18          const user = await this.users.findById(payload.sub);
19          if (!user) {
20              throw new UnauthorizedException();
21          }
22          // Vai trò lấy từ database mỗi lần gọi API, nên đổi quyền có
    hiệu lực ngay.
23          return { id: user.id, email: user.email, role: user.role };
24      }
```

Hình 21: JWT strategy lấy vai trò hiện tại từ User trong database

backend/src/auth/roles.guard.ts · lines 16–28

```
16      canActivate(context: ExecutionContext): boolean {
17          const required = this.reflector.getAllAndOverride<Role[] |
    undefined>(
18              ROLES_KEY,
19              [context.getHandler(), context.getClass()],
20          );
21          if (!required || required.length === 0) return true;
22
23          const { user } = context.switchToHttp().getRequest<{ user?:
    AuthUser }>();
24          if (!user || !required.includes(user.role)) {
25              throw new ForbiddenException('Tài khoản này không có quyền
    thực hiện thao tác này');
26          }
27          return true;
28      }
```

Hình 22: RolesGuard đối chiếu vai trò với metadata của route

chữ ký như mã hóa nội dung hoặc bằng chứng rằng đơn chắc chắn còn được phục vụ. QueryDr ký theo chuỗi trường của giao thức riêng, không dùng lại querystring của URL thanh toán.

5.4.1.2 Transaction và idempotency. apply khóa Order trước, đọc lại attempt và kiểm paidAt/khoản đã áp dụng. Với kết quả thành công, chỉ canFulfill khi đơn payable còn hạn, số tiền khớp và snapshot còn giữ. Payment, Order PAID, điểm và nhật ký cùng transaction; lỗi ở bước cộng điểm rollback cả settlement. Key của Payment theo txnRef, unique providerTxnNo và liên kết attempt là các lớp kiểm soát bổ sung; khóa Order điều phối cả hai attempt khác nhau của cùng đơn.

5.4.1.3 Không giữ khóa trong lúc chờ mạng. QueryDr thực hiện ngoài transaction, có timeout 8 giây. lastCheckedAt được cập nhật có điều kiện để hạn chế tra cứu dồn trong khoảng 15 giây; đây là điều tiết truy vấn, không phải kết quả tài chính. Khi response được xác minh, apply lấy khóa và kiểm lại điều kiện tại thời điểm ghi. HTTP 200 của endpoint IPN chỉ xác nhận HTTP có phản hồi; mã RspCode trong JSON mới là ACK nghiệp vụ, không thay thế trạng thái Order/Payment.

5.5 Toàn vẹn đơn hàng, thanh toán và tồn kho

5.5.1 Order State Machine

Bảng chuyển trạng thái hợp lệ nằm trong một tệp riêng, không phụ thuộc cơ sở dữ liệu nên kiểm thử được độc lập.

```
export const ORDER_TRANSITIONS = {
  PENDING: ['PAID', 'PAYMENT_FAILED', 'CANCELLED'],
  PAYMENT_FAILED: ['PENDING', 'PAID', 'CANCELLED'],
  PAID: ['PREPARING', 'CANCELLED'],
  PREPARING: ['READY'],
  READY: ['COMPLETED'],
  COMPLETED: [],
  CANCELLED: [],
};

export function assertTransition(from, to) {
  if (!canTransition(from, to)) {
    throw new ConflictException('Khong the chuyen don tu ' + from + ' sang ' + to);
  }
}
```

5.5.2 Thanh toán idempotent

Mỗi yêu cầu thanh toán kèm header `Idempotency-Key`. Cùng một khóa chỉ được xử lý một lần; các lần sau trả lại kết quả của lần đầu, nên khách bấm nhiều lần hoặc mạng chập chờn cũng không ghi nhận khoản thanh toán hai lần bởi cùng một yêu cầu. Toàn bộ thanh toán chạy trong một giao dịch cơ sở dữ liệu. Dòng đơn hàng được khóa bằng `SELECT ... FOR UPDATE` để hai yêu cầu cùng một đơn phải xếp hàng, nhờ đó cổng thanh toán chỉ bị gọi một lần. Cột `idempotencyKey` duy nhất trong bảng Payment là chốt chặn cuối cùng.

```
await tx.$queryRaw`SELECT id FROM "Order"
                WHERE id = ${dto.orderId} FOR UPDATE`;
const replay = await this.findReplay(tx, userId, key, dto.orderId);
if (replay) return replay; // cung Idempotency-Key: tra ket qua cu
```

backend/src/payments/vnpay.crypto.ts · lines 12–22

```
12  export function canonical(fields: Fields): string {
13      const query = new URLSearchParams();
14      for (const key of Object.keys(fields).sort()) {
15          if (key !== 'vnp_SecureHash' && key !== 'vnp_SecureHashType')
    query.append(key, fields[key]);
16      }
17      return query.toString();
18  }
19  export function signedQuery(fields: Fields, secret: string): string {
20      const data = canonical(fields);
21      return `${data}&vnp_SecureHash=${hmac(secret, data)}`;
22  }
```

Hình 23: Chuẩn hóa tham số và ký URL trong vnpay.crypto.ts

backend/src/payments/vnpay.service.ts · lines 104–113

```
104     async returnUrl(input: Record<string, unknown>) {
105         const config = this.gateway.config();
106         const fields = readCallback(input, config.secret);
107         if (!fields || fields.vnp_TmnCode !== config.merchant) throw new
    BadRequestException('Không xác minh được URL trả về');
108         const attempt = await this.prisma.vnpayAttempt.findUnique({
    where: { txnRef: fields.vnp_TxnRef || '' } });
109         if (!attempt || attempt.merchantCode !== fields.vnp_TmnCode
110             || fields.vnp_Amount !== String(attempt.amount * 100)) throw
    new BadRequestException('Thông tin trả về không khớp');
111         // Return chỉ điều hướng; tuyệt đối không ghi PAID tại đây.
112         return `${config.frontend}/orders/${attempt.orderId}?
    vnpay=returned`;
113     }
```

Hình 24: Return kiểm chứng dữ liệu rồi chỉ tạo đích chuyển hướng

backend/src/payments/vnpay.service.ts · lines 161–166

```
161                 if (canFulfill) {
162                     await tx.payment.create({ data: { orderId: order.id,
    amount: attempt.amount, method: 'VNPAY',
163                         idempotencyKey: `vnpay:${attempt.txnRef}`,
    vnpayAttemptId: attempt.id } });
164                     await tx.order.update({ where: { id: order.id },
    data: { status: 'PAID' } });
165                     await this.loyalty.award(tx, order.userId,
    attempt.amount, order.id);
166                 }
```

Hình 25: Nhánh canFulfill ghi Payment, PAID và điểm trong transaction apply

5.5.3 Kiểm soát tồn kho khi đặt đồng thời

Khi nhiều khách cùng đặt món sử dụng chung nguyên liệu, backend phải bảo đảm tổng lượng giữ thành công không vượt tồn kho. Hệ thống tính nhu cầu theo size/topping, khóa các dòng Ingredient theo thứ tự id tăng dần, kiểm tra đủ lượng rồi mới trừ trong cùng transaction tạo Order và snapshot. Khách gửi lại cùng yêu cầu không bị giữ kho lần hai; hủy hợp lệ hoặc hết thời hạn giữ món sẽ hoàn đúng lượng đã lưu, không tính lại từ công thức hiện tại. Các phần dưới trình bày trách nhiệm từng hàm, cơ chế phối hợp và ảnh code hiện thực.

5.5.3.1 Tổ chức các hàm và ranh giới transaction.

Hàm/thành phần	Trách nhiệm
OrdersService.create	Xác lập quyền sở hữu idempotency key, tính giá/voucher, gọi reserve, tạo Order/OrderItem/snapshot và gắn kết quả vào key trong một transaction.
ingredientDemand	Chuyển lựa chọn món/size/topping thành tổng lượng theo ingredientId; kiểm tra định lượng và giới hạn số nguyên.
InventoryService.reserve	Đọc công thức, tính demand, khóa Ingredient theo id tăng dần, kiểm đủ toàn bộ rồi trừ; trả lượng đã trừ để lưu snapshot.
OrdersService.cancel	Kiểm tra quyền/trạng thái, khóa Order, kiểm tra thanh toán, cập nhật CANCELLED có điều kiện và gọi release.
InventoryService.release	Khóa snapshot theo đơn; chặn thiếu snapshot/đã hoàn; cộng lại quantity và ghi releasedAt trong transaction do hàm gọi cung cấp.
ReservationExpiryService	Tìm đơn quá hạn đủ điều kiện, lấy khóa không chờ trên đơn, kiểm lại sau khóa, release và đóng đơn; phát thông báo sau commit.
ExchangeService.decide	Khi duyệt đổi, khóa Order cũ và tập nguyên liệu liên quan, hoàn snapshot cũ, giữ lựa chọn mới và tạo đơn thay thế trong cùng transaction.

Bảng 27: Phân trách nhiệm để tái sử dụng và kiểm thử độc lập

5.5.3.2 Tính nguyên tử. Giả sử đơn có cà phê và trà: cà phê còn đủ nhưng trà hết. Backend không được tạo một nửa Order hoặc giữ riêng phần cà phê. Transaction làm cho thao tác hoặc commit toàn bộ hoặc rollback toàn bộ. Tương tự, hủy đơn không được commit trạng thái CANCELLED nhưng bỏ quên bước hoàn nguyên liệu. Snapshot, kho và trạng thái phải cùng commit.

5.5.3.3 Chống gửi lặp trước khi chạm kho. Unique key theo khách trong OrderCreateRequest được xác lập trước reserve. Hai request cùng key cùng nội dung sẽ quy về một Order; key cũ với nội dung khác bị từ chối. Vô hiệu hóa nút frontend chỉ hỗ trợ thao tác, còn ràng buộc database mới bảo vệ khi request bị gửi lại hoặc có nhiều tab/process. Khi một transaction tranh key thất bại, backend rollback rồi đọc kết quả đã lưu, không tiếp tục truy vấn trong transaction đang lỗi.

5.5.3.4 Chống tranh phần cuối bằng khóa dòng. Reserve dùng SELECT FOR UPDATE trên các Ingredient cần dùng. Nếu A và B cùng cần 20g cuối, A lấy khóa trước, kiểm đủ rồi trừ về 0 và commit. B chờ khóa; khi được xử lý, B thấy tồn 0 nên bị từ chối. Khóa chỉ giữ trong transaction ngắn; không giữ trong lúc khách chọn món hoặc mở trang VNPay. Database còn có CHECK stock không âm như lớp bảo vệ dữ liệu.

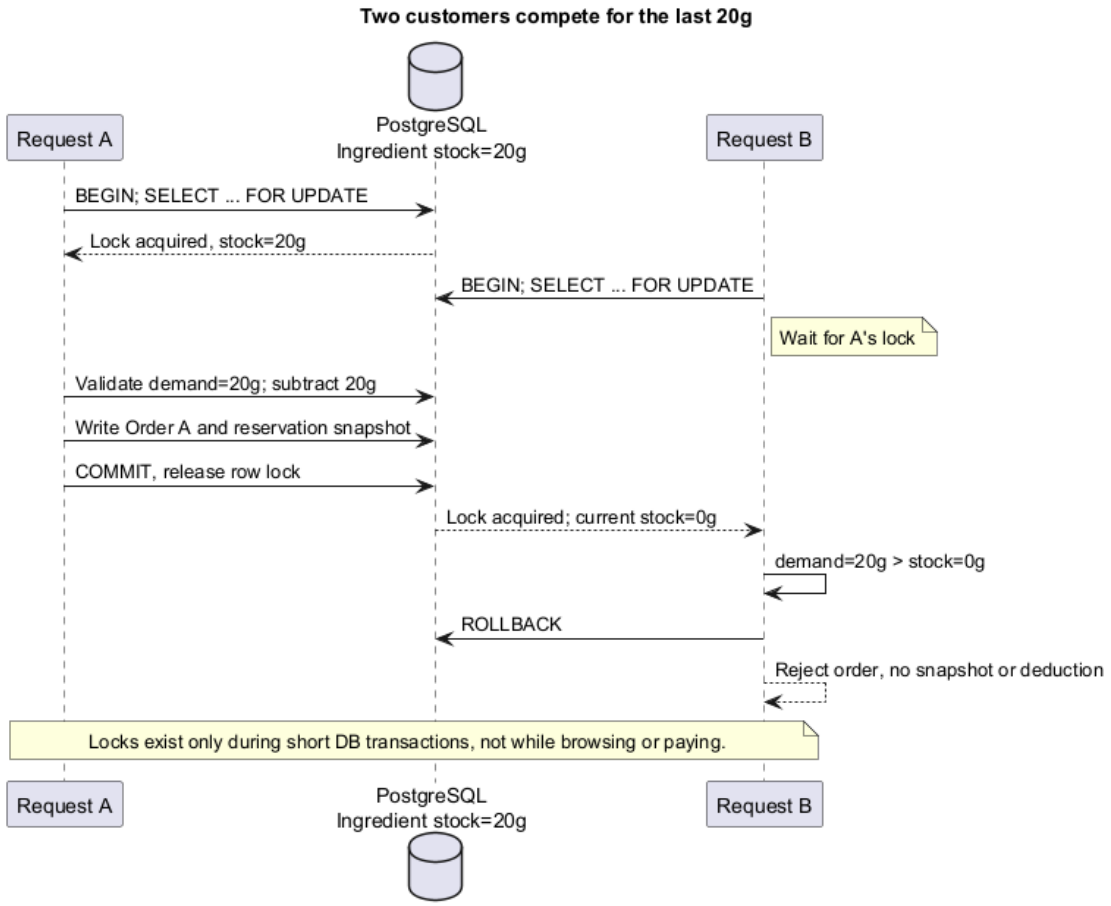
Khóa theo id tăng dần giúp các transaction lấy nhiều nguyên liệu theo cùng thứ tự. Ví dụ cả A và B đều lấy nguyên liệu 1 rồi 2, tránh tình huống mỗi bên giữ một nguyên liệu rồi chờ nguyên liệu của bên kia. Các luồng thanh toán/hủy/hết hạn cùng lấy khóa Order trước để kiểm tra trạng thái nhất quán.

5.5.3.5 Hoàn đúng một lần. Hai yêu cầu hủy cùng Order phải qua khóa Order và cập nhật trạng thái có điều kiện. Chỉ giao dịch thực sự chuyển được trạng thái mới release. Bên trong release, snapshot tiếp tục được khóa và releasedAt được kiểm tra. Hai lớp này bảo vệ cả API hủy lẫn việc gọi lại hàm release; lỗi sau khi cộng kho vẫn rollback cả dấu hoàn.

5.5.3.6 Nhiều worker và thông báo. Cờ running tránh hai lượt quét chồng trong một process. SELECT FOR UPDATE SKIP LOCKED mới là cơ chế phối hợp ở database khi nhiều worker cùng xử lý một đơn: bên chưa lấy được khóa bỏ qua, không xử lý trùng. Sau khi lấy khóa, worker đọc lại trạng thái và bằng chứng nhận tiền. Thông báo orders.changed phát sau commit; UI tải lại dữ liệu và có polling HTTP 15 giây khi tab hiển thị. Tác vụ quét tiếp tục khi backend khởi động lại vì deadline nằm trong database.

5.5.3.7 Kết hợp khóa dòng và kiểm tra phiên bản. Kho và vòng đời Order dùng khóa dòng. OrderExchange và PaymentRefundCase dùng thêm cập nhật có điều kiện theo version: chỉ ghi khi id/version/trạng thái còn khớp, rồi tăng version; cập nhật không tác động một dòng trả lỗi xung đột. Ví dụ hai barista cùng đọc version=0 thì người ghi sau không thể ghi đè quyết định đã nâng lên version=1. Mỗi cơ chế gắn với đối tượng nghiệp vụ cụ thể.

5.5.3.8 Ảnh chụp code từ phiên bản hiện thực. Các hình sau được chụp từ mã nguồn hiện tại; tên tệp và số dòng nằm trên ảnh để truy vết. Phần giải thích đọc cùng các hàm gọi bên ngoài: reserve/release nhận TransactionClient, nên ranh giới commit/rollback do service gọi quản lý.



Hình 26: Hai khách tranh phần nguyên liệu cuối: chỉ transaction đủ kho được commit

backend/src/products/ingredient-demand.ts · lines 22–39

```
22      for (const item of items) {
23          const product = products.find((p) => p.id === item.productId);
24          if (!product?.recipes.length)
25              throw new ConflictException(`${product?.name ?? 'Món'} chưa
    có công thức nên chưa bán được`);
26          if (!SIZE_PERCENT[item.size] || !Number.isInteger(item.qty) ||
    item.qty < 1)
27              throw new ConflictException('Lựa chọn món không hợp lệ');
28          for (const line of product.recipes) {
29              const perCup = line.scalesWithSize
30                  ? Math.ceil(line.quantity * SIZE_PERCENT[item.size] /
    100) : line.quantity;
31              add(line.ingredientId, perCup * item.qty);
32          }
33          for (const code of new Set(item.toppings ?? [])) {
34              const recipe = toppingRecipes.filter((line) =>
    line.toppingCode === code);
35              if (!recipe.length) throw new ConflictException(`Topping
    ${code} chưa có công thức, vui lòng bỏ chọn`);
36              // Một phần topping trên mỗi ly, không nhân thêm theo size.
37              for (const line of recipe) add(line.ingredientId,
    line.quantity * item.qty);
38          }
39      }
```

Hình 27: ingredientDemand: tính size, vật tư cố định, topping và cộng gộp nguyên liệu

backend/src/products/inventory.service.ts · lines 35–60

```
35          const ids = [...demand.keys()].sort((a, b) => a - b);
36          const rows = await tx.$queryRaw<{ id: number; stock: number }
    []>`
37              SELECT id, stock FROM "Ingredient"
38              WHERE id IN (${Prisma.join(ids)})
39              ORDER BY id
40              FOR UPDATE`;
41          const stockById = new Map(rows.map((r) => [r.id, r.stock]));
42
43          for (const id of ids) {
44              if ((stockById.get(id) ?? 0) < (demand.get(id) as number)) {
45                  throw new ConflictException('Không đủ nguyên liệu cho
    size/topping đã chọn, vui lòng chọn lại món');
46              }
47          }
48
49          for (const id of ids) {
50              await tx.ingredient.update({
51                  where: { id },
52                  data: { stock: { decrement: demand.get(id) as number }
    },
53              });
54          }
55          // Lưu chính lượng vừa trừ, không đọc công thức thêm lần nữa để
    tạo snapshot.
56          return ids.map((ingredientId) => ({
57              ingredientId,
58              quantity: demand.get(ingredientId) as number,
59          }));
60      }
```

Hình 28: reserve: khóa dòng theo thứ tự id, kiểm kho, trừ và trả snapshot

backend/src/products/inventory.service.ts · lines 63–97

```
63      async release(tx: Prisma.TransactionClient, orderId: number) {
64          const reservations = await tx.$queryRaw<{
65              ingredientId: number;
66              quantity: number;
67              releasedAt: Date | null;
68          }[]>`
69              SELECT "ingredientId", quantity, "releasedAt"
70              FROM "OrderInventoryReservation"
71              WHERE "orderId" = ${orderId}
72              ORDER BY "ingredientId"
73              FOR UPDATE`;
74
75          if (reservations.length === 0) {
76              // Đơn cũ không có bằng chứng lượng đã trừ. Không suy ngược
    từ Recipe.
77              throw new ConflictException(
78                  'Đơn chưa có dữ liệu giữ nguyên liệu, cần nhân viên đối
    soát trước khi hủy',
79              );
80          }
81          if (reservations.some((row) => row.releasedAt !== null)) {
82              throw new ConflictException('Nguyên liệu của đơn đã được
    hoàn');
83          }
84
85          for (const row of reservations) {
86              await tx.ingredient.update({
87                  where: { id: row.ingredientId },
88                  data: { stock: { increment: row.quantity } },
89              });
90          }
91          await tx.orderInventoryReservation.updateMany({
92              where: { orderId },
93              data: { releasedAt: new Date() },
94          });
95      }
96
97  }
```

Hình 29: release: khóa snapshot, chặn hoàn lại và cộng đúng lượng lịch sử

backend/src/orders/reservation-expiry.service.ts · lines 23–41

```
23      async expireOne(id: number, now = new Date()): Promise<boolean> {
24          const changed = await this.prisma.$transaction(async (tx) => {
25              const locked = await tx.$queryRaw<{ id: number }[]>`
26                  SELECT id FROM "Order" WHERE id = ${id} FOR UPDATE SKIP
    LOCKED`;
27              if (!locked.length) return false;
28              const row = await tx.order.findUniqueOrThrow({ where: { id }
    });
29              if (!['PENDING', 'PAYMENT_FAILED'].includes(row.status) ||
    row.creditApplied > 0 ||
30                  !row.reservationExpiresAt || row.reservationExpiresAt >
    now || row.reservationExpiredAt) return false;
31              // Không đóng đơn đã có bằng chứng nhận tiền đang chờ nhân
    viên xử lý.
32              if (await tx.payment.findFirst({ where: { orderId: id } })
    ||
33                  await tx.vnpayAttempt.findFirst({ where: { orderId: id,
    paidAt: { not: null } } })) return false;
34              await this.inventory.release(tx, id);
35              await tx.order.update({ where: { id }, data: { status:
    'CANCELLED', reservationExpiredAt: now } });
36              // KHÔNG biến attempt chưa rõ thành FAILED: IPN muộn phải
    được ghi REVIEW/refund case.
37              return true;
38          }, { timeout: 15000 });
39          if (changed) this.events.notifyOrdersChanged();
40          return changed;
41      }
```

Hình 30: expireOne: phối hợp worker, kiểm tra bằng chứng thanh toán và đóng đơn nguyên tử

5.5.4 Khuyến mãi và điểm thưởng

PromotionsService.findValid dùng được với PrismaService khi xem trước và TransactionClient khi đặt hàng. calcDiscount/calcPoints là hàm thuần, tách quy tắc số học khỏi database để kiểm thử độc lập. OrdersService tạo dòng món theo giá backend, tính subtotal, kiểm voucher rồi lưu total/discount cùng đơn và snapshot kho trong một transaction.

LoyaltyService.award nhận cùng TransactionClient của thanh toán. Khi points dương, UPDATE User dùng increment nguyên tử, lấy số dư mới rồi ghi LoyaltyTransaction EARN. Payment, PAID, số dư và sổ điểm cùng commit hoặc cùng rollback; gửi lại kết quả đã xử lý không được gọi lại để cộng điểm thêm. Unique sổ điểm là lớp bảo vệ bổ sung, không thay thế kiểm settlement dưới khóa Order.

Hai đơn khác nhau cùng trả cho một User dùng phép increment tránh ghi đè số dư đọc cũ. Đổi bằng giá chỉ chuyển credit, không gọi award; đổi cao hơn gọi award qua thanh toán phần chênh lệch. Frontend đọc lại thông tin tài khoản sau kết quả thanh toán để hiển thị điểm do backend trả.

5.6 Hiện thực đổi món sau thanh toán

ExchangeService.quote/quoteIn tính báo giá từ dữ liệu backend; request kiểm quyền, trạng thái, yêu cầu chờ và key trước khi lưu báo giá. withdraw rút yêu cầu bằng trạng thái/version. decide thực hiện quyết định của quầy. Hàng đợi hỗ trợ lọc trạng thái, tìm kiếm và phân trang; frontend ExchangeWorkspace giữ giỏ đổi riêng, không trộn với lần mua thông thường.

5.6.1 Một quyết định, một transaction

decide khóa Order nguồn trước, đọc lại OrderExchange và kiểm version/REQUESTED. Khi duyệt, kiểm nguồn còn PAID và tiền đã áp dụng khớp; dùng quotedLines để giữ thành tiền đã chấp nhận. Tiếp theo khóa tập hợp nguyên liệu cũ và mới theo id tăng dần, hoàn snapshot cũ, reserve lựa chọn mới theo size/topping, tạo Order thay thế và snapshot, chuyển credit rồi đóng Order nguồn. Cuối cùng cập nhật quyết định, tăng version và commit. Thiếu nguyên liệu mới hoặc lỗi bất kỳ làm rollback cả bước hoàn cũ.

5.6.2 Tranh chấp với pha chế và thao tác quầy

Luồng bắt đầu pha và quyết định đổi cùng khóa Order. API bắt đầu pha kiểm yêu cầu REQUESTED; API duyệt kiểm nguồn PAID. Vì vậy thao tác dựa trên trạng thái cũ bị chặn sau khi trạng thái đã thay đổi. UPDATE yêu cầu có điều kiện id/version/status và kiểm count=1 bảo vệ trước thao tác đã cũ; version tăng sau khi duyệt, từ chối hoặc rút. Thông báo UI phát sau commit.

5.6.3 Bảo toàn tiền đã chuyển

Đơn thay thế có credit không đi qua tự hủy/hết hạn thông thường của đơn chưa từng trả tiền. Các thao tác giữ/hoàn tài nguyên tái sử dụng cơ chế mục 5.5.3.1; thu chênh lệch tái sử dụng VNPay mục 5.4.1. Không cộng credit thành một Payment mới, vì việc đó sẽ làm sai thống kê dòng tiền và có thể cộng điểm trùng.

backend/src/promotions/promotions.service.ts · lines 15–23

```
15          if (!voucher) {
16              throw new BadRequestException('Mã giảm giá không tồn tại');
17          }
18          if (!voucher.active) {
19              throw new BadRequestException('Mã giảm giá đã ngừng áp
    dụng');
20          }
21          if (voucher.expiresAt <= now) {
22              throw new BadRequestException('Mã giảm giá đã hết hạn');
23          }
```

Hình 31: findValid kiểm mã, hiệu lực và hạn trước khi áp dụng

backend/src/promotions/discount.ts · lines 1–11

```
1  export const POINT_UNIT = 10000;
2
3  export function calcDiscount(subtotal: number, percent: number): number
   {
4      const safePercent = Math.min(Math.max(percent, 0), 100);
5      const discount = Math.floor((subtotal * safePercent) / 100);
6      return Math.min(discount, subtotal);
7  }
8
9  export function calcPoints(amount: number): number {
10     return Math.max(0, Math.floor(amount / POINT_UNIT));
11 }
```

Hình 32: Hai hàm thuần tính tiền giảm và điểm với quy tắc làm tròn

backend/src/promotions/loyalty.service.ts · lines 24–33

```
24              await tx.loyaltyTransaction.create({
25                  data: {
26                      userId,
27                      orderId,
28                      type: 'EARN',
29                      points,
30                      balanceAfter: user.loyaltyPoints,
31                  },
32              });
33          }
```

Hình 33: award ghi EARN cùng số dư sau cập nhật trong transaction thanh toán

backend/src/orders/exchange.service.ts · lines 189–193

```
189           quotedTotal: quote.total,
190           quotedLines: quote.lines as Prisma.InputJsonValue,
191           items: items as unknown as Prisma.InputJsonValue,
192           reason,
193         },
```

Hình 34: request lưu báo giá và nội dung đã được khách xác nhận

backend/src/orders/exchange.service.ts · lines 443–454

```
443           await this.inventory.release(tx, source.id);
444           const reservations = await this.inventory.reserve(tx, items);
445           const replacement = await tx.order.create({
446             data: {
447               userId: source.userId,
448               total,
449               creditApplied: received,
450               status: difference === 0 ? "PAID" : "PENDING",
451               items: { create: lines },
452               inventoryReservations: { create: reservations },
453             },
454           });
```

Hình 35: Hoàn snapshot cũ, giữ lựa chọn mới và tạo đơn chuyển credit trong transaction

backend/src/orders/exchange.service.ts · lines 459–471

```
459           const changed = await tx.orderExchange.updateMany({
460             where: { id, version, status: "REQUESTED" },
461             data: {
462               status: "APPROVED",
463               version: { increment: 1 },
464               replacementOrderId: replacement.id,
465               decidedBy: actorId,
466               decidedAt: new Date(),
467               decisionNote: note,
468             },
469           });
470           if (changed.count !== 1)
471             throw new ConflictException("Yêu cầu vừa thay đổi");
```

Hình 36: Quyết định cập nhật có điều kiện theo version và trạng thái REQUESTED

5.7 Hiện thực quầy và cập nhật theo thời gian thực

5.7.1 Kiểm soát chuyển trạng thái tại backend

StaffOrdersController nhận PATCH trạng thái qua JwtAuthGuard/RolesGuard; DTO chỉ nhận PREPARING, READY hoặc COMPLETED. OrdersService.updateStaffStatus khóa Order, đọc lại trạng thái, kiểm yêu cầu đổi chờ và gọi assertTransition. UPDATE kèm điều kiện trạng thái hiện tại, kiểm count trước khi commit. Hai request cùng duyệt một đơn PAID sang PREPARING chỉ có một request hợp lệ; request tiếp theo không được coi PREPARING sang PREPARING là bước mới.

Máy trạng thái là luật chuyển tiếp dùng chung, còn quyền và điều kiện cụ thể nằm ở service. Có cạnh PAID sang CANCELLED không có nghĩa API quầy được hủy tùy ý: luồng đổi món/hủy/hết hạn có kiểm tra riêng. Các phần xử lý tài chính và kho được tham chiếu ở những mục tương ứng, không thực hiện lại khi bấm "Pha xong" hoặc "Đã giao khách".

5.7.2 Thông báo sau commit và tải lại dữ liệu

OrdersGateway dùng namespace /orders-live, phát orders.changed không payload. notifyOrdersChanged được gọi sau transaction thành công; rollback hoặc replay không làm một kết quả chưa commit xuất hiện như thành công trên màn hình khác.

Hook useOrdersLive đăng ký connect/disconnect/connect_error và orders.changed. Khi kết nối hoặc nhận sự kiện, client tải lại API. Có polling HTTP 15 giây khi tab hiển thị và nút tải lại để hỗ trợ khi mất kênh tức thời. Mỗi refresh hủy request trước bằng AbortController; cleanup gỡ listener, ngắt socket và dọn interval khi component rời màn hình. Thanh trạng thái kết nối chỉ biểu diễn kênh cập nhật, không chứng minh một đơn đã trả tiền hoặc đã pha.

5.8 Đóng gói và triển khai

Trong môi trường phát triển, PostgreSQL chạy qua Docker Compose; backend và frontend chạy ở hai terminal riêng. Việc đóng gói dịch vụ gắn với cấu hình mạng, biến môi trường và kiểm tra trạng thái sẵn sàng. Seed chạy chủ động cho dữ liệu mẫu; migration áp dụng có kiểm soát để bảo toàn dữ liệu.

backend/src/orders/orders.service.ts · lines 280–291

```
280       assertTransition(order.status, status);
281
282       // Hai nhân viên cùng thao tác: chỉ người còn đúng trạng thái cũ
    mới ghi được.
283       const result = await tx.order.updateMany({
284         where: { id, status: order.status },
285         data: { status },
286       });
287       if (result.count === 0) {
288         throw new ConflictException(
289           "Đơn vừa được cập nhật ở màn hình khác, vui lòng tải lại",
290         );
291       }
```

Hình 37: Cập nhật có điều kiện sau khi kiểm luật chuyển tiếp và khóa Order

backend/src/realtime/orders.gateway.ts · lines 13–16

```
13      notifyOrdersChanged() {
14          // Không đưa đơn/tài khoản lên kênh chung. Client gọi lại API
    của mình.
15          this.server?.emit('orders.changed');
16      }
```

Hình 38: Gateway chỉ phát tín hiệu thay đổi, không đưa dữ liệu đơn lên kênh chung

frontend/src/features/orders/useOrdersLive.ts · lines 18–26

```
18          request.current?.abort();
19          const controller = new AbortController();
20          request.current = controller;
21          setIsLoading(true);
22          try {
23              await load(controller.signal);
24          } finally {
25              if (!controller.signal.aborted) setIsLoading(false);
26          }
```

Hình 39: Client chỉ giữ request tải mới và hủy request cũ khi refresh

6 Kiểm thử và kết quả

6.1 Chiến lược kiểm thử

Nhóm dùng hai loại kiểm thử tự động bằng Vitest:

- **Test đơn vị** (`npm run test`): kiểm tra các hàm thuần như state machine và tính giảm giá, chạy trong vài giây và không cần cơ sở dữ liệu.
- **Test tích hợp API** (`npm run test:e2e`): dựng cả ứng dụng NestJS và gọi API thật bằng Supertest trên PostgreSQL thật. Mỗi test tự tạo người dùng, sản phẩm và đơn tạm rồi xóa đi sau khi chạy, không đụng vào dữ liệu mẫu.

Test API tập trung vào hợp đồng và dữ liệu; browser E2E kiểm tra thao tác xuyên suốt từ trình duyệt tới backend và database. Dùng cơ sở dữ liệu kiểm thử riêng. Các lệnh test chạy trong thư mục `backend`, và test end-to-end cần cơ sở dữ liệu đang chạy.

6.2 Kiểm thử các quy tắc toàn vẹn nghiệp vụ

Yêu cầu	Tệp test	Nội dung kiểm tra
(a) Chặn chuyển trạng thái sai	`order-state-machine.spec.ts` và các ca hủy đơn trong `payments.e2e-spec.ts`	Các bước nhảy cóc như PENDING sang READY bị chặn; COMPLETED và CANCELLED không đi tiếp; hủy đơn hai lần và thanh toán đơn đã hủy bị từ chối
(b) Gửi 2 lần cùng `Idempotency-Key` chỉ tạo 1 đơn	`order-idempotency.e2e-spec.ts` và `payments.e2e-spec.ts`	Gửi liên tiếp và gửi đồng thời cùng khóa chỉ tạo một Order ở test tạo đơn; test thanh toán chỉ tạo 1 dòng Payment và cùng một `paymentId`
(c) Đặt đồng thời không vượt tồn kho	`inventory.e2e-spec.ts`	10 đơn đồng thời cho món còn 3 phần thì đúng 3 đơn thành công, 7 đơn bị từ chối, kho về 0
Khuyến mãi và điểm thưởng	`discount.spec.ts`, `promotions.e2e-spec.ts`	Tính giảm giá và điểm đúng; mã sai, hết hạn hoặc đã tắt bị từ chối; điểm chỉ cộng một lần

Bảng 28: Ánh xạ giữa quy tắc nghiệp vụ và các test

6.3 Danh sách test case tự động

Tệp test	Các ca kiểm tra chính
`order-state-machine.spec.ts`	Luồng thành công PENDING, PAID, PREPARING, READY, COMPLETED; thanh toán lỗi rồi thử lại; 9 ca chuyển trạng thái sai bị chặn; hai trạng thái cuối không đi tiếp; danh sách chuyển tiếp hợp lệ khớp đúng bảng luật
`discount.spec.ts`	Giảm 10% của 120.000đ là 12.000đ; làm tròn xuống; không giảm quá tổng tiền; phần trăm không dương thì không giảm; dưới 10.000đ chưa được điểm; 108.000đ được 10 điểm

Tệp test	Các ca kiểm tra chính
payments.e2e-spec.ts	Gửi hai lần liên tiếp và đồng thời cùng khóa; cùng khóa cho đơn khác bị 409; thanh toán lỗi rồi thử lại; thanh toán lại đơn PAID bị 409; thiếu header bị 400; hủy đơn hai lần và đơn đã thanh toán không hủy được
inventory.e2e-spec.ts	10 đơn đồng thời cho món còn 3 phần; 6 đơn số lượng 2 cho món còn 5 phần; đơn lớn hơn tồn kho bị từ chối; hai dòng cùng món được cộng dồn; đơn nhiều dòng thiếu một món bị hoàn tác toàn bộ; hủy đơn hoàn kho đúng một lần
promotions.e2e-spec.ts	Mã hợp lệ không phân biệt hoa thường; mã không tồn tại, hết hạn, đã tắt bị 400; đơn có mã giảm đúng 10%; mã sai thì không tạo đơn và không trừ kho; thanh toán đúng số tiền sau giảm và cộng điểm đúng một lần

Bảng 29: Các test tự động

6.4 Kiểm chứng luồng đặt hàng, giữ kho và hoàn kho

Kiểm thử tách thành ba lớp: hàm tính định lượng thuần; API tích hợp với PostgreSQL để kiểm tra transaction/khóa/ràng buộc; giao diện để kiểm tra lựa chọn, thông báo và điều hướng. Dữ liệu thử được tạo riêng để kết quả có thể kiểm tra trực tiếp từ tồn kho, snapshot và trạng thái đơn.

Ca	Thiết lập và thao tác	Điều kiện kiểm tra
KHO-01	Hai ly M, cơ bản 18g, topping dùng chung 5g, một ly nhựa mỗi ly.	Giữ 56g và hai ly; không nhân ly nhựa theo hệ số size.
KHO-02	Topping làm vượt nguyên liệu dù món cơ bản còn đủ.	Backend trả xung đột; Order và kho không đổi. Availability đúng theo tổ hợp.
KHO-03	Mười đơn đồng thời, kho đủ ba phần.	Đúng ba đơn thành công, bảy đơn bị từ chối; kho về 0.
KHO-04	Hai món khác nhau cùng tranh nguyên liệu cuối đủ một ly L.	Chỉ một đơn thành công, không coi mỗi Product có một kho riêng.
KHO-05	Một đơn nhiều dòng có một nguyên liệu thiếu.	Không lưu đơn một phần, không giữ kho của các dòng còn lại.
KHO-06	Gửi đồng thời cùng key khi còn một phần.	Trả cùng một Order, một lần trừ và một bộ snapshot.
KHO-07	Đặt giữ 40g rồi sửa công thức hoặc thay nguyên liệu.	Hủy vẫn hoàn đúng 40g vào nguyên liệu cũ.
KHO-08	Hai yêu cầu hủy đồng thời, sau đó gọi release lại.	Chỉ một lần hoàn; lần sau bị chặn, releasedAt không tạo thêm stock.
KHO-09	Cố ý ném lỗi sau reserve hoặc sau release.	Rollback cả kho, snapshot và trạng thái Order.
KHO-10	Khách khác hủy đơn; đơn PAID dùng API hủy thường.	Bị từ chối, kho và trạng thái không đổi.
KHO-11	Đơn quá hạn, hai worker cùng xử lý.	Một lần đóng/hoàn, có reservationExpiredAt và thông báo đặt lại.

Ca	Thiết lập và thao tác	Điều kiện kiểm tra
KHO-12	Đã qua deadline nhưng worker chưa chạy; gửi yêu cầu thanh toán.	Không tạo khoản thanh toán mới cho đơn quá hạn.
KHO-13	IPN thành công đến sau deadline, trước hoặc sau worker.	Không PAID/pha chế; REVIEW và một hồ sơ khoản thu dư; kho được bảo toàn.
KHO-14	QR cũ hết hạn nhưng đơn còn thời gian giữ.	Cấp QR mới; expiry QR không vượt deadline Order, không giữ kho lần hai.
KHO-15	Đơn đã trả tiền hoặc đã dùng credit; worker quét.	Không tự đóng như đơn bỏ thanh toán thông thường.

Bảng 30: Các ca kiểm chứng và kết quả mong đợi của quản lý nguyên liệu

Các test được tổ chức theo trách nhiệm:

- `inventory.e2e-spec.ts`: cạnh tranh tồn kho, đầu vào và hủy đơn.
- `ingredients.e2e-spec.ts`: công thức và nguyên liệu dùng chung.
- `inventory-snapshot.e2e-spec.ts`: công thức thay đổi, hoàn một lần và rollback.
- `order-idempotency.e2e-spec.ts`: gửi lặp và tranh key tạo đơn.
- `size-topping-expiry.e2e-spec.ts`: định lượng lựa chọn, deadline, worker và IPN muộn.

Hàm tính định lượng và chính sách deadline/QR có unit test riêng. Các ca ném lỗi chủ động kiểm tra rollback, vì vậy thông báo lỗi trong log phải được đọc cùng assertion và kết quả PASS của test, không chỉ dựa vào màu log.

6.4.1 Kết quả kiểm chứng theo phạm vi

Ngày 12/11/2026, bản vá 13 độc lập được dựng trong staging và chạy trên database thử mới, tách khỏi dữ liệu vận hành. Kết quả ghi nhận gồm 42/42 unit test (7 tệp) và 135/135 API integration test (12 tệp). Các con số là tổng bộ hồi quy của bản vá, bao gồm cả thanh toán, quyền và đổi món; không phải 135 ca chỉ riêng tồn kho. TypeScript backend/frontend, build backend và build production frontend đều đạt trong môi trường kiểm chứng đó.

Ở lần kiểm tra source đã ghép, bộ unit test hiện có ghi nhận 31/31 ca đạt. Hai kết quả có phạm vi bộ test khác nhau và được ghi riêng; kết quả của staging không được quy đổi thành kết quả nghiệm thu toàn bộ nhánh tích hợp. Hồ sơ kiểm chứng của bản vá nằm tại `proposed-fixes/13-size-topping-reservation/VALIDATION.md`.

Kiểm tra giao diện dùng Edge headless, kích thước desktop 1280×900 và mobile 390×844, với API fixture để cô lập trạng thái màn hình. Kiểm tra gồm thông báo hết hạn trong chi tiết/lịch sử, CTA đặt lại, không có thao tác hủy trên đơn đã đóng, topping không đủ hàng bị khóa và nút thêm giỏ bị vô hiệu khi chọn tổ hợp hết nguyên liệu. Ảnh dưới là minh chứng UI với dữ liệu thử; kiểm thử API PostgreSQL ở trên mới kiểm tra tính đúng của database và transaction.

6.5 Kiểm chứng thanh toán VNPay

Unit test kiểm HMAC bằng vector RFC 4231, chuẩn hóa khoảng trắng/ký tự đặc biệt, callback sửa tiền/trường trùng, giờ GMT+7 và chữ ký QueryDr. API integration dùng gateway giả lập có chữ ký test cùng PostgreSQL riêng để chủ động tạo callback lặp, trễ và lỗi giữa transaction.



Hình 40: Giao diện thông báo hết thời gian giữ món và yêu cầu đặt lại (API fixture)

Mã	Hành vi cần chứng minh
PAY-01	Backend quyết định amount/merchant; cùng key hoặc key mới tái dùng mã còn hiệu lực.
PAY-02	IPN thành công đồng thời chỉ có một Payment và một lần cộng điểm.
PAY-03	Return hợp lệ chỉ redirect, không tự chuyển Order sang PAID.
PAY-04	Chữ ký, merchant hoặc amount sai không cập nhật đơn.
PAY-05	Timeout tra cứu không tự đánh dấu thất bại; retry sau hết hạn giữ đủ lịch sử attempt.
PAY-06	Hai khoản thu cùng đơn chỉ một khoản áp dụng, khoản dư có hồ sơ xử lý.
PAY-07	Khách không xem giao dịch người khác/gọi API staff; bật VNPay thì API mock bị chặn.
PAY-08	Thất bại đến sau thành công không hạ PAID; lỗi cộng điểm rollback, IPN retry có thể xử lý lại.

Bảng 31: Các ca kiểm chứng chữ ký, vòng đời và tính nhất quán thanh toán

Các ca tương ứng nằm tại `src/payments/vnpay.crypto.spec.ts`, `test/vnpay.e2e-spec.ts` và bộ retry/đổi món. Các kiểm tra hạn QR và IPN sau deadline liên kết với KHO-12 đến KHO-14. Số liệu bộ hồi quy đã ghi tại mục 6.4; không cộng lại thành một kết quả chạy mới hoặc diễn giải gateway giả lập là giao dịch tiền thật.



Hình 41: Màn hình QR và nút mở trang VNPay; ảnh UI dùng API fixture với tham chiếu thử

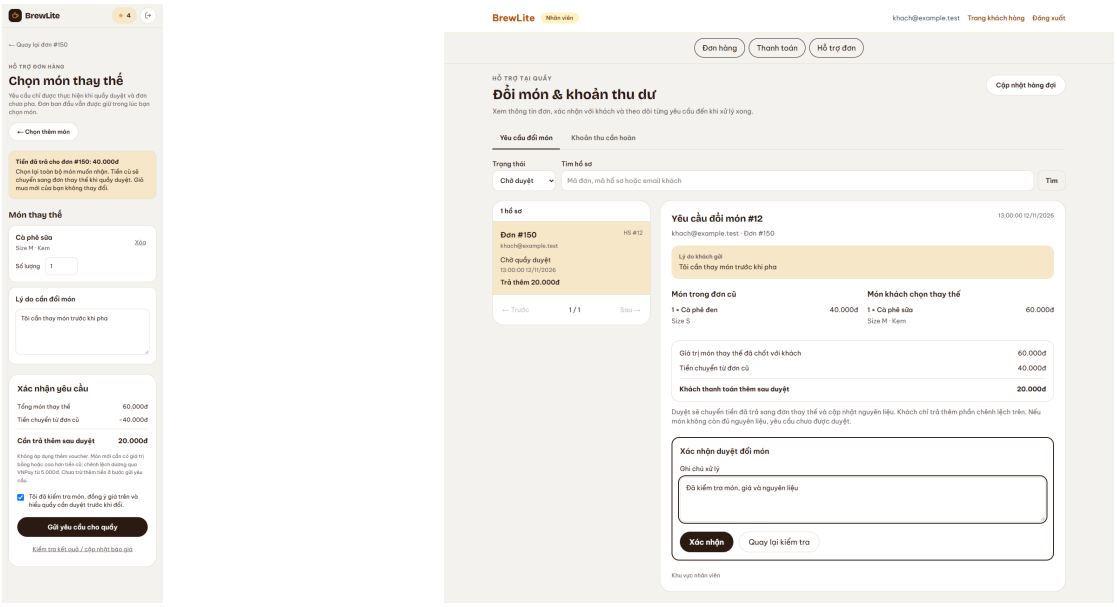
6.6 Kiểm chứng đổi món và thu chênh lệch

Các kịch bản có tại `test/payment-exchange.e2e-spec.ts`, thuộc bộ hồi quy đã nêu ở mục 6.4; không suy ra một kết quả chạy mới chỉ từ việc bổ sung tài liệu. Kiểm thử giao diện của bản vá

Ca	Điều kiện cần chứng minh
EX-01	Đổi bằng giá chuyển credit, hoàn/giữ snapshot đúng, không tạo Payment giả.
EX-02	Đổi cao hơn chỉ thu delta qua VNPay, điểm chỉ tăng theo tiền mới.
EX-03	Món thấp hơn giá trị đã trả bị yêu cầu chọn lại; thiếu kho mới rollback toàn bộ.
EX-04	Đơn PREPARING không được đổi; khách không gọi API quyết định của quầy.
EX-05	Duyệt đổi và bắt đầu pha đồng thời không cùng thành công theo trạng thái cũ.
EX-06	Giá đổi trước gửi làm fingerprint cũ bị từ chối; giá đổi sau gửi không thay báo giá đã lưu.
EX-07	REQUESTED chặn pha; rút yêu cầu đúng version cho phép tiếp tục đơn cũ, version cũ bị chặn.
EX-08	Hàng đợi lọc/phân trang và quyền BARISTA được kiểm ở backend.

Bảng 32: Các ca kiểm chứng đổi món trong bộ payment-exchange

08 sử dụng fixture để kiểm chọn món/giỏ đổi, gửi yêu cầu và thao tác quầy. Ảnh dưới minh họa UI với dữ liệu thử, không phải bằng chứng thu tiền thật.

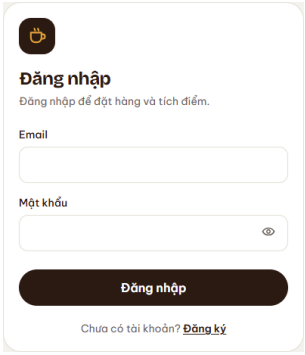


Hình 42: Giỏ đổi của khách và màn hình xử lý của barista (ảnh kiểm thử bằng fixture)

6.7 Kiểm chứng xác thực và phân quyền

Ba ca đầu được hiện thực trong `test/auth-roles.e2e-spec.ts`. Kiểm tra staff, chủ đơn và giao dịch được phân bố trong các bộ staff-orders, inventory-snapshot và VNPay. AUTH-06 mô tả tiêu chí của cấu hình strategy kiểm chữ ký/hạn; bảng tiêu chí không thay thế log chạy test. Kết quả bộ hồi quy được trình bày theo phạm vi tại mục 6.4, không bổ sung số liệu chạy mới ở phần này.

Ca	Tiêu chí kiểm chứng
AUTH-01	Đăng ký gửi kèm role BARISTA vẫn tạo CUSTOMER, không tự nâng quyền.
AUTH-02	Login và /auth/me trả vai trò đúng, không có passwordHash.
AUTH-03	Cùng token, đổi role trong database làm quyền gọi /staff/orders thay đổi ở request tiếp theo.
AUTH-04	Route staff thiếu token trả 401; CUSTOMER có token bị 403.
AUTH-05	Khách khác không đọc/hủy/check giao dịch của chủ đơn; dữ liệu và kho không bị thay đổi bởi yêu cầu bị chặn.
AUTH-06	Tiêu chí xác thực token: token sai chữ ký hoặc hết hạn không được đi vào nghiệp vụ được bảo vệ.



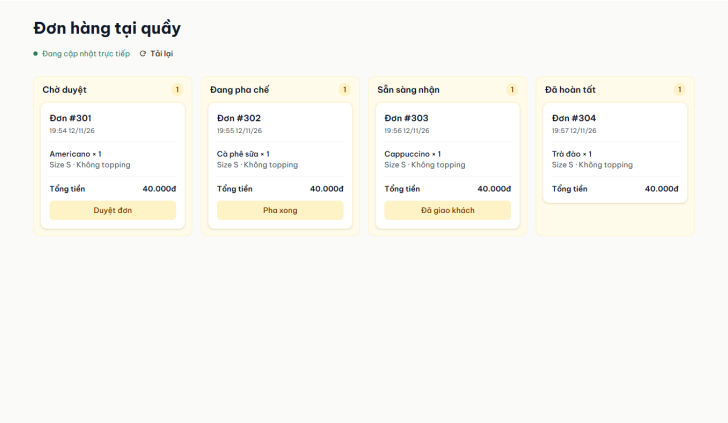
Bảng 33: Tiêu chí kiểm chứng quyền và form đăng nhập chụp từ frontend, không có thông tin tài khoản thật

6.8 Kiểm chứng mã giảm giá và điểm thưởng

Unit test nằm tại `src/promotions/discount.spec.ts`; API test nằm trong promotions, loyalty, staff-orders, VNPay và payment-exchange. Các bộ này kiểm đồng thời kết quả phản hồi, User.loyaltyPoints và dòng sổ EARN. Phạm vi kết quả đã xác minh dùng chung với mục 6.4; phần này không bổ sung một số liệu chạy mới. Ảnh giỏ hàng chỉ kiểm chứng hiển thị giảm giá với dữ liệu thử.

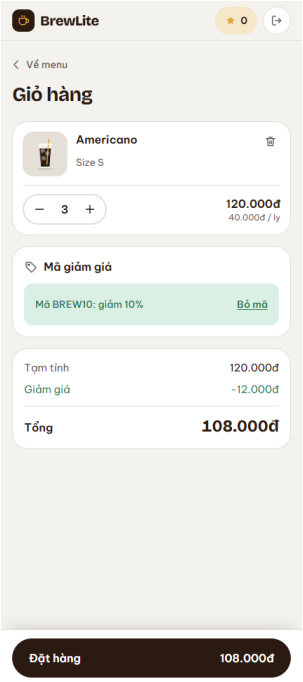
6.9 Kiểm chứng thao tác quầy và realtime

Các ca có tại `test/staff-orders.e2e-spec.ts`; bộ này có client Socket.IO kết nối thật tới ứng dụng test và đọc PostgreSQL để kiểm thứ tự commit/notify. Đây không chỉ là giả lập gọi một hàm emit. Các số liệu bộ hồi quy tham chiếu mục 6.4; ảnh dưới sử dụng API fixture để minh họa bảng quầy, không tạo một kết quả chạy backend mới.



Hình 43: Bốn cột xử lý đơn tại quầy với dữ liệu thử

Ca	Nội dung kiểm tra
KM-01	Chuẩn hóa hoa/thường, khoảng trắng; mã thiếu/hết hạn/inactive bị từ chối.
KM-02	120.000đ giảm 10% còn 108.000đ; làm tròn đúng và không giảm vượt tổng.
KM-03	Mã sai không tạo đơn hoặc trừ nguyên liệu.
KM-04	PAID ghi số dư và một EARN khớp amount sau giảm; dưới ngưỡng không ghi EARN.
KM-05	Request/IPN lặp không nhân đôi điểm; unique chặn EARN trùng.
KM-06	Lỗi cộng điểm rollback settlement; đổi món chỉ cộng điểm trên tiền mới.



Bảng 34: Tiêu chí kiểm chứng khuyến mãi/điểm và giỏ hàng áp mã với API fixture

Ca	Điều kiện kiểm tra
QUAY-01	Chỉ BARISTA vào staff API; không hiển thị đơn chưa trả/hủy trong hàng đợi pha.
QUAY-02	Đi đúng PAID–PREPARING–READY–COMPLETED; không tạo thêm Payment, trừ kho hoặc cộng điểm.
QUAY-03	Bỏ bước, lùi bước, bấm lặp và trạng thái/đầu vào sai bị từ chối.
QUAY-04	Hai nhân viên cùng duyệt một đơn: một response 200, một 409, trạng thái cuối nhất quán.
QUAY-05	Hai client WebSocket nhận sự kiện không payload sau commit, đọc API thấy trạng thái mới.
QUAY-06	Rollback và replay không phát thông báo thành công mới; hủy chỉ báo sau khi kho đã hoàn.
QUAY-07	Kết nối lại đọc được trạng thái mới; quyền đọc đơn cá nhân không bị mở rộng.
QUAY-08	Hàng đợi hơn 20 đơn vẫn giữ toàn bộ PAID/PREPARING/READY; nhóm hoàn tất giới hạn riêng.

Bảng 35: Các ca API và WebSocket cho quy trình quầy

6.10 Tiêu chí kiểm chứng menu, tùy chọn và giỏ hàng

Ca	Điều kiện kiểm chứng
GIO-01	Menu tải dữ liệu, có trạng thái chờ/rỗng/lỗi; ảnh trống hoặc lỗi có minh họa dự phòng.
GIO-02	Chọn size/topping cập nhật giá; lựa chọn thiếu nguyên liệu không được thêm theo UI, backend kiểm lại khi đặt.
GIO-03	Cùng món/size/topping gộp qty dù thứ tự topping khác; khác tùy chọn tách dòng.
GIO-04	Đổi qty/xóa cập nhật tổng và badge; tải lại vẫn có giỏ, không gây lệch HTML trước hydration.
GIO-05	Chưa đăng nhập đi qua login; gửi order không tin giá client và chỉ xóa giỏ khi đã nhận kết quả tạo đơn.
GIO-06	Mất phản hồi dùng lại checkout key; kiểm tra idempotency chứng minh không tạo thêm Order hoặc giữ thêm kho.

Bảng 36: Tiêu chí chấp nhận luồng khách chọn món và đặt hàng

Các ca frontend kiểm ở trình duyệt; phần backend tính lại giá, kiểm input và idempotency có test API trong inventory/order-idempotency. Bảng là tiêu chí kiểm chứng, không thay thế log của một lần chạy browser E2E. Ảnh giao diện trong mục 4.11 minh họa các màn hình; việc chụp không gửi tạo Order hoặc thanh toán.

6.11 Kiểm thử chức năng thủ công

Mã	Thao tác	Kết quả mong đợi
TC01	Mở trang chủ	Hiện lưới 4 món có giá; tắt backend thì hiện dòng báo lỗi
TC02	Mở chi tiết Cappuccino, chọn size L và Trân châu	Thành tiền 60.000đ
TC03	Thêm cùng món cùng size hai lần	Số lượng tăng thành 2, không thêm dòng mới; tải lại trang giỏ vẫn còn
TC04	Đăng ký trùng email	Báo lỗi "Email đã được đăng ký"
TC05	Đăng nhập sai mật khẩu	Báo lỗi chung "Email hoặc mật khẩu không đúng"
TC06	Bấm "Đặt hàng" ở giỏ hàng	Tạo đơn PENDING, giỏ hàng về trống, tồn kho giảm
TC07	Thanh toán đơn PENDING bằng Ví	Đơn chuyển PAID, hiện "Đã nhận thanh toán, chờ quầy xác nhận"
TC08	Thanh toán với tùy chọn giả lập lỗi	Đơn chuyển PAYMENT_FAILED, vẫn thanh toán lại hoặc hủy được
TC09	Hủy đơn PENDING	Đơn chuyển CANCELLED, tồn kho được hoàn lại
TC10	Đơn đã PAID	Không có nút hủy; gọi API hủy trả lỗi 409
TC11	Nhập mã BREW10 với giỏ 120.000đ	Giảm 12.000đ, tổng 108.000đ; thanh toán xong được 10 điểm
TC12	Nhập mã không tồn tại	Báo lỗi "Mã giảm giá không tồn tại"

Bảng 37: Các ca kiểm thử chức năng thủ công

7 Hướng dẫn triển khai

7.1 Chạy bằng Docker

Ở thư mục gốc của dự án (cần mở Docker Desktop):

```
docker compose up -d db
```

Lệnh trên hiện chỉ chạy PostgreSQL ở cổng 5432; backend và frontend phải chạy ở hai terminal riêng. Kiểm tra cổng và DATABASE_URL nếu máy có nhiều PostgreSQL. Dừng bằng `docker compose down`; không dùng tùy chọn xóa volume khi cần giữ dữ liệu.

7.2 Chạy chế độ phát triển

Terminal backend, từ thư mục gốc:

```
cd backend
npm.cmd install
npx.cmd prisma generate
npx.cmd prisma migrate status
npm.cmd run start:dev
```

Sao lưu trước khi áp dụng thay đổi cấu trúc; dùng `npx.cmd prisma migrate deploy` cho các migration tăng dần và `npx.cmd prisma generate` để đồng bộ client. Với database mẫu mới, chủ động chạy `npx.cmd prisma db seed`. Quá trình triển khai bảo toàn dữ liệu và không reset database vận hành.

Terminal frontend, mở mới tại thư mục gốc:

```
cd frontend
npm.cmd install
npm.cmd run dev
```

Địa chỉ phát triển: web http://localhost:3000, API mặc định http://localhost:3001; PORT và cấu hình thực tế có thể thay đổi. Ngrok cho callback Sandbox phải trỏ tới backend, Return URL và IPN URL phải đúng route được khai báo.

7.3 Biến môi trường

Biến	**Nơi dùng**	**Ý nghĩa**
DATABASE_URL	backend	Chuỗi kết nối PostgreSQL
JWT_SECRET	backend	Khóa ký token đăng nhập
NEXT_PUBLIC_API_URL	frontend	Địa chỉ backend mà trình duyệt gọi
PORT, FRONTEND_URL	backend	Cổng API và địa chỉ frontend để chuyển hướng sau thanh toán
VNPAY_*	backend	Mã merchant, khóa ký, URL Sandbox, Return/IPN theo cấu hình; không đưa khóa thật vào báo cáo

Bảng 38: Các biến môi trường

7.4 Dữ liệu mẫu

Hệ thống có sẵn 4 sản phẩm (Cà phê sữa 35.000đ, Americano 40.000đ, Cappuccino 45.000đ, Trà đào 39.000đ, tồn kho theo Ingredient và định lượng Recipe, không phải 50 phần cố định cho mỗi món) và mã giảm giá BREW10 giảm 10%. Tài khoản được tạo ở trang đăng ký.

8 Đánh giá và hướng phát triển

8.1 Giá trị của thiết kế

Việc tách công thức, định lượng lựa chọn và snapshot giúp hệ thống đồng thời phục vụ bán hàng và bảo toàn tài nguyên lịch sử. Transaction cùng khóa dòng bảo vệ phần nguyên liệu cuối; idempotency bảo vệ request gửi lại; version kiểm soát cập nhật hồ sơ nghiệp vụ. Deadline bền vững phối hợp giữ món với thanh toán và cung cấp thông báo rõ cho khách.

8.2 Định hướng mở rộng

Các hướng mở rộng gồm cấu hình định lượng theo từng sản phẩm/size, công cụ nhập và kiểm kê nguyên liệu, phân tích nhu cầu tiêu thụ, quan sát tác vụ nền và tối ưu trải nghiệm đặt lại. Nghiệp vụ giao hàng được xem xét trong phạm vi vận hành cửa hàng, còn báo cáo tập trung vào đặt và nhận món tại quầy.

9 Kết luận

BrewLite đã có các luồng chọn món, đặt đơn, thanh toán giả lập/VNPay Sandbox, nghiệp vụ quầy và đổi món. Báo cáo trình bày yêu cầu sản phẩm, thiết kế, hiện thực, kiểm thử và hướng dẫn triển khai để phục vụ bàn giao và bảo trì. Các quy tắc được liên kết với mô hình dữ liệu, hàm hiện thực và kịch bản kiểm thử, giúp giải thích từ nhu cầu nghiệp vụ tới quyết định kỹ thuật.

Tài liệu

[1] VNPAY, *Kết nối cổng thanh toán và API QueryDr*, https://sandbox.vnpayment.vn/apis/docs/thanh-toan-pay/pay.html, truy cập ngày 12/11/2026.

[2] VNPAY, *Câu hỏi thường gặp về tham số và Return/IPN*, https://sandbox.vnpayment.vn/apis/docs/faqs/, truy cập ngày 12/11/2026.

[3] NestJS Documentation, "**link: https://docs.nestjs.com**", *NestJS - A progressive Node.js framework*, lần truy cập cuối: 11/2026.

[4] Next.js Documentation, "**link: https://nextjs.org/docs**", *Next.js by Vercel*, lần truy cập cuối: 11/2026.

[5] Prisma Documentation, "**link: https://www.prisma.io/docs**", *Prisma ORM*, lần truy cập cuối: 11/2026.

[6] Vitest Documentation, "**link: https://vitest.dev**", *Vitest - Next generation testing framework*, lần truy cập cuối: 11/2026.

[7] Docker Documentation, "**link: https://docs.docker.com**", *Docker and Docker Compose*, lần truy cập cuối: 11/2026.

[8] Ken Schwaber và Jeff Sutherland, "**link: https://scrumguides.org**", *The Scrum Guide*, lần truy cập cuối: 11/2026.